



Static Factorisation of Probabilistic Programs with User-Labelled Sample Statements and While Loops

MARKUS BÖCK, TU Wien, Austria

JÜRGEN CITO, TU Wien, Austria

It is commonly known that any Bayesian network can be implemented as a probabilistic program, but the reverse direction is not so clear. In this work, we address the open question to what extent a probabilistic program with user-labelled sample statements and while loops – features found in languages like Gen, Turing, and Pyro – can be represented graphically. To this end, we extend existing operational semantics to support these language features. By translating a program to its control-flow graph, we define a sound static analysis that approximates the dependency structure of the random variables in the program. As a result, we obtain a static factorisation of the implicitly defined program density, which is equivalent to the known Bayesian network factorisation for programs without loops and constant labels, but constitutes a novel graphical representation for programs that define an unbounded number of random variables via loops or dynamic labels. We further develop a sound program slicing technique to leverage this structure to statically enable three well-known optimisations for the considered program class: we reduce the variance of gradient estimates in variational inference and we speed up both single-site Metropolis Hastings and sequential Monte Carlo. These optimisations are proven correct and empirically shown to match or outperform existing techniques.

CCS Concepts: • **Theory of computation** → **Operational semantics; Program analysis**; • **Mathematics of computing** → *Bayesian computation; Bayesian networks*.

Additional Key Words and Phrases: Probabilistic programming, operational semantics, static program analysis, factorisation, Bayesian networks

ACM Reference Format:

Markus Böck and Jürgen Cito. 2026. Static Factorisation of Probabilistic Programs with User-Labelled Sample Statements and While Loops. *Proc. ACM Program. Lang.* 10, OOPSLA1, Article 115 (April 2026), 28 pages. <https://doi.org/10.1145/3798223>

1 Introduction

Probabilistic programming provides an intuitive means to specify probabilistic models as programs. Typically, probabilistic programming systems also come with machinery for automatic posterior inference. This enables practitioners without substantial knowledge in Bayesian inference techniques to work with complex probabilistic models.

Besides these practical aspects, there is also a long history of studying probabilistic programming languages (PPLs) from a mathematical perspective [2, 32]. It is commonly known that any Bayesian network [31] can be implemented as a probabilistic program [23], but the reverse direction is not so clear. The understanding is that first-order probabilistic programs without loops can be compiled to Bayesian networks, because the number of executed sample statements is fixed and finite, such that each sample statement corresponds to exactly one random variable [8, 52, 57]. Generally, a graphical representation of a probabilistic program is desirable as it lends itself for optimising

Authors' Contact Information: Markus Böck, TU Wien, Vienna, Austria, markus.h.boeck@tuwien.ac.at; Jürgen Cito, TU Wien, Vienna, Austria, juergen.cito@tuwien.ac.at.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2475-1421/2026/4-ART115

<https://doi.org/10.1145/3798223>

inference algorithms by exploiting the dependency structure between random variables. In fact, many probabilistic programming languages require the user to explicitly construct the model in a graphical structure [37, 40, 41, 43, 51].

However, in the universal probabilistic programming paradigm probabilistic constructs are embedded in a Turing-complete language [21]. While loops and recursive function calls easily lead to an unbounded number of executed sample statements which makes the typical compilation to a graph with one node per random variable impossible. Even more, many PPLs allow the user to label each sample statement with a dynamic address. This further complicates matters, because in this case even a single sample statement may correspond to a potentially infinite number of random variables. As a consequence, many approaches resort to dynamic methods to leverage the dependency structure of this kind of probabilistic programs for efficient inference [12, 59, 60].

Sample statements with dynamic address expressions constitute a valuable language feature, as they allow the definition of random variables whose identifiers differ from the program variables to which values are assigned to. This permits the specification of a broader model class relative to PPLs that lack this feature. Below we give examples of such user-labelled sample statements in Gen [17], Pyro [6], and Turing [20], where the address expressions are highlighted with red:

```
x = {:x => i} ~ Normal(0., 1.)           # Gen
x = pyro.sample(f"x{i}", Normal(0., 1.)) # Pyro
x[i] ~ Normal(0., 1.)                   # Turing
x = sample("x"+str(i), Normal(0., 1.))   # our formal syntax
```

In this work, we address the open question to what extent a probabilistic program with *user-labelled sample statements and while loops* can be represented graphically. We prove that the density of a probabilistic program with K user-labelled sample statements decomposes into K factors. This finite factorisation holds even if the program contains while loops. Moreover, the dependency structure implied by the factorisation can be computed *statically* which, as we will show, enables new opportunities for the optimisation of inference algorithms.

1.1 Overview

Section 2. Address-Based Semantics For Probabilistic Programs generalises the operational semantics of Core Stan [24]. Instead of interpreting the program density p as a function over only a fixed set of finite variables, we extend this approach to interpret the density as a function over traces. In contrast to Gorinova et al. [24], this allows us to consider programs with dynamic addresses and while loops. These language features greatly expand the class of expressible probabilistic models, for instance, models with an infinite number of random variables. The described semantics can be seen as an operational analog to the denotational models of Pyro [33] and Gen [35].

Section 3. Static Factorisation for Programs Without Loops examines the factorisation of the program density for the simplified case where programs do not contain while loops.

In Section 3.2, building on standard data and control flow analysis techniques, we present a static provenance analysis that finds the dependencies of a program variable x at any program state σ . The dependencies are described by specifying for which addresses a trace tr needs to be known to also know the value of x in σ . This analysis forms the basis for finding the factorisation over addresses and operates on the control-flow graph (CFG) of a program.

In Section 3.3, we derive a novel factorisation from the static analysis, $p(\text{tr}) = \prod_{k=1}^K p_k(\text{tr})$, for programs without loops by explicitly constructing one factor p_k per sample statement. If we further restrict the program to constant addresses, we arrive at the language of Gorinova et al. [24], where our factorisation theorem implies a known equivalence to Bayesian networks. However, we show

that already in the case of dynamic addresses, programs are only equivalent to Markov networks, because of the possibility of cyclic dependencies and an infinite number of random variables.

Section 4. Static Factorisation for Programs With Loops describes a way to generalise the factorisation result of Section 3 to the much more challenging case where programs may contain while loops. The idea is to mathematically unroll while loops to obtain a directed acyclic graph, the *unrolled CFG*, that essentially represents all possible program paths. Our main theoretical contribution is to show that even in this case, the density still factorises into a finite number of terms – one factor for each of the K sample statements. Like dynamic addresses, while loops may also be used to model an infinite number of random variables and thus, the resulting factorisation in general implies the equivalence of programs to Markov networks.

Section 5. Optimising Posterior Inference Algorithms presents a novel and sound program slicing method building on the factorisation theory established in Section 4. By generating sub-programs for each factor, we can 1) decrease the runtime of a Metropolis Hastings algorithm by speeding up the acceptance probability computation; 2) construct a lower-variance gradient estimator for Black-Box Variational Inference; 3) facilitate an iterative implementation of sequential Monte Carlo. Each of the considered optimisations are known, but have been either realised dynamically or were reserved for PPLs that require the explicit graphical construction of probabilistic models. Our approach is the first to enable them statically for the considered program class. We evaluate our approach on a benchmark set of 16 probabilistic programs, the majority of which make use of while loops to define an unbounded number of random variables.

1.2 Contributions

We briefly summarise the key contributions of this work:

- A static provenance analysis in the context of our formal PPL building on standard data and control flow techniques,
- A novel density factorisation for the considered program class, which can be computed statically and subsumes a known equivalence to Bayesian networks in a restricted setting,
- A novel and sound program slicing method derived from the factorisation theory,
- An application of the slicing method to enable well-known optimisations *statically and provably correct* for the considered program class and three inference algorithms: Metropolis Hastings, variational inference, and sequential Monte Carlo,
- An open-source implementation and empirical evaluation of our proposed approach [10].

2 Address-Based Semantics for Probabilistic Programs

First, we introduce the syntax of our formal probabilistic programming language in terms of expressions and statements similar to Gorinova et al. [24] and Hur et al. [30]. We assume that the constants of this language range over a set of values including booleans, integers, real numbers, immutable real-valued vectors, finite-length strings, and a special null value. We define $\mathcal{V}$ to be the set of all values. A program contains a finite set of program variables denoted by x , x_i , y , or z . We further assume a set of built-in functions g . We assume that functions g are total and return null for erroneous inputs. The only non-standard construct in the language are sample statements. For sample statements, the expression E_0 corresponds to the user-defined sample address, which is assumed to evaluate to a string. The symbol f ranges over a set of built-in distributions which are parameterised by arguments $E_1, \dots, E_n$.

Syntax of Expressions:

$E ::=$	expression
c	constant
x	variable
$g(E_1, \dots, E_n)$	function call

Syntax of Statements:

$S ::=$	statement
$x = E$	assignment
$S_1; S_2$	sequence
if E then S_1 else S_2	if statement
skip	skip
while E do S	while loop
$x = \text{sample}(E_0, f(E_1, \dots, E_n))$	sample

2.1 Operational Semantics

The meaning of a program is the density implicitly defined by its sample statements. This density is evaluated for a *program trace* and returns a real number. A program trace is a mapping from finite-length addresses to numeric values, $\text{tr}: \text{Strings} \rightarrow \mathcal{V}$. We denote the set of all traces with $\mathcal{T}$. How the value at an address is determined is not important for this work and in practice typically depends on the algorithm that is used to perform posterior inference for the program. For instance, if an address α corresponds to an observed variable, the value $\text{tr}(\alpha)$ is fixed to the data. If an address α does not appear in a program, then its value can be assumed to be $\text{tr}(\alpha) = \text{null}$.

For each trace tr , the operational semantics of our language are defined by the big-step relation $(\sigma, S) \Downarrow^{\text{tr}} \sigma'$, where S is a statement and $\sigma \in \Sigma$ is a program state – a finite map from program variables to values

$$\sigma ::= x_1 \mapsto V_1, \dots, x_n \mapsto V_n, \quad x_i \text{ distinct, } V_i \in \mathcal{V}.$$

A state can be naturally lifted to a map from expressions to values $\sigma: \text{Exprs} \rightarrow \mathcal{V}$:

$$\sigma(x_i) = V_i, \quad \sigma(c) = c, \quad \sigma(g(E_1, \dots, E_n)) = g(\sigma(E_1), \dots, \sigma(E_n)).$$

The operational semantics for the non-probabilistic part of our language are standard:

$$\begin{array}{c} \frac{}{(\sigma, \text{skip}) \Downarrow^{\text{tr}} \sigma} \quad \frac{}{(\sigma, x = E) \Downarrow^{\text{tr}} \sigma[x \mapsto \sigma(E)]} \quad \frac{(\sigma, S_1) \Downarrow^{\text{tr}} \sigma' \quad (\sigma', S_2) \Downarrow^{\text{tr}} \sigma''}{(\sigma, S_1; S_2) \Downarrow^{\text{tr}} \sigma''} \\[10pt] \frac{\sigma(E) = \text{true} \quad (\sigma, S_1) \Downarrow^{\text{tr}} \sigma'}{(\sigma, \text{if } E \text{ then } S_1 \text{ else } S_2) \Downarrow^{\text{tr}} \sigma'} \quad \frac{\sigma(E) = \text{false} \quad (\sigma, S_2) \Downarrow^{\text{tr}} \sigma'}{(\sigma, \text{if } E \text{ then } S_1 \text{ else } S_2) \Downarrow^{\text{tr}} \sigma'} \\[10pt] \frac{\sigma(E) = \text{false}}{(\sigma, \text{while } E \text{ do } S) \Downarrow^{\text{tr}} \sigma} \quad \frac{\sigma(E) = \text{true} \quad (\sigma, (S; \text{while } E \text{ do } S)) \Downarrow^{\text{tr}} \sigma'}{(\sigma, \text{while } E \text{ do } S) \Downarrow^{\text{tr}} \sigma'} \end{array}$$

Above, $\sigma[x \mapsto V]$ denotes the updated program state σ' , where $\sigma'(x) = V$ and $\sigma'(y) = \sigma(y)$ for all other variables $y \neq x$. The inference rules are to be interpreted inductively such that the semantics of a while loop can be rewritten as

$$(\sigma, \text{while } E \text{ do } S) \Downarrow^{\text{tr}} \sigma' \iff \exists n \in \mathbb{N}_0: ((\sigma, \text{repeat}_n(S)) \Downarrow^{\text{tr}} \sigma' \wedge \sigma'(E) = \text{false}) \wedge \forall m < n: ((\sigma, \text{repeat}_m(S)) \Downarrow^{\text{tr}} \sigma'_m \wedge \sigma'_m(E) = \text{true}),$$

where $\text{repeat}_n(S) = (S; \dots; S)$ repeated n -times. In particular, if a while loop does not terminate for state σ then $\nexists \sigma': (\sigma, \text{while } E \text{ do } S) \Downarrow^{\text{tr}} \sigma'$. This makes the presented operational semantics partial, where errors and non-termination are modelled implicitly. Formally, $\Downarrow^{\text{tr}}$ is the least-relation closed under the stated rules.

At sample statements, we inject the value of the trace at address $V_0 = \sigma(E_0)$. We multiply the density, represented by the reserved variable $\mathbf{p}$, with the value of function pdf_f evaluated at $\text{tr}(V_0)$.

Note that the function pdf_f may be an arbitrary function to real values. However, in the context of probabilistic programming, we interpret pdf_f as the density function of distribution f with values in $\mathbb{R}_{\geq 0}$. Lastly, the value $\text{tr}(V_0)$ is stored in the program state at variable x . The semantics of sample statements are summarised in the following rule:

$$\frac{\forall i: \sigma(E_i) = V_i \wedge V_i \neq \text{null} \quad V_0 \in \mathbf{Strings} \quad V = \text{tr}(V_0) \wedge V \neq \text{null}}{(\sigma, x = \text{sample}(E_0, f(E_1, \dots, E_n))) \Downarrow^{\text{tr}} \sigma[x \mapsto V, \mathbf{p} \mapsto \sigma(\mathbf{p}) \cdot \text{pdf}_f(V; V_1, \dots, V_n)]} \quad (1)$$

Finally, the meaning of a program S is defined as the mapping of trace to density. This definition is well-defined, because the semantics are deterministic for each trace. If a trace leads to errors or non-termination, then the density is undefined.

DEFINITION 1. *The density of a program S is the function $\bar{p}_S: \mathcal{T} \times \Sigma \rightarrow \mathbb{R}_{\geq 0}$ given by*

$$\bar{p}_S(\text{tr}, \sigma_0) := \begin{cases} \sigma(\mathbf{p}) & \text{if } \exists \sigma \in \Sigma: (\sigma_0, S) \Downarrow^{\text{tr}} \sigma \\ \text{undefined} & \text{otherwise} \end{cases}$$

For the initial program state $\sigma_0 = (x_1 \mapsto \text{null}, \dots, x_n \mapsto \text{null}, \mathbf{p} \mapsto 1)$, mapping all program variables to null, we define $p_S: \mathcal{T} \rightarrow \mathbb{R}_{\geq 0}$ with $p_S(\text{tr}) = \bar{p}_S(\text{tr}, \sigma_0)$.

```
p = sample("p", Uniform(0,1))
x = sample("x", Bernoulli(p))
if x == 1 then
  y = sample("y", Bernoulli(0.25))
else
  z = sample("z", Bernoulli(0.75))
```

Listing 1. Simple probabilistic program with stochastic branching.

```
p = sample("p", Uniform(0,1))
x = sample("x", Bernoulli(p))
if x == 1 then
  addr = "y"; prob = 0.25;
else
  addr = "z"; prob = 0.75;
r = sample(addr, Bernoulli(prob))
```

Listing 2. Program of Listing 1 rewritten with dynamic addresses.

Consider the simple probabilistic program in Listing 1 (Note that we slightly deviate from the formal syntax in listings for better readability). The density defined by our operational semantics is given below, where δ_v denotes the delta function whose value is 1 if v equals true and 0 otherwise.

$$\begin{aligned} p(\text{tr}) = & \text{pdf}_{\text{Uniform}}(\text{tr}(\text{"p"}); 0, 1) \\ & \cdot \text{pdf}_{\text{Bernoulli}}(\text{tr}(\text{"x"}); \text{tr}(\text{"p"})) \\ & \cdot (\delta_{\text{tr}(\text{"x"})} \text{pdf}_{\text{Bernoulli}}(\text{tr}(\text{"y"}); 0.25) + (1 - \delta_{\text{tr}(\text{"x"})})) \\ & \cdot (\delta_{\text{tr}(\text{"x"})} + (1 - \delta_{\text{tr}(\text{"x"})}) \text{pdf}_{\text{Bernoulli}}(\text{tr}(\text{"z"}); 0.75)). \end{aligned}$$

As the addresses in Listing 1 are constant and identical to the program variables, the program is well-described by many existing PPL semantics. However, in Listing 2, we rewrite the program with *dynamic addresses* which results in the same above density. Most formal PPLs studied in prior work do not support user-defined addresses for sample statements and cannot interpret a program with three sample statements as a four-dimensional joint density. In contrast, address-based semantics like those of Lee et al. [33] and Lew et al. [35], denotationally modelling Pyro and Gen, respectively, correctly interpret the program with dynamic addresses. This is also true for our extensions of Gorinova et al. [24]. While closely related to the aforementioned denotational models, we opted for an operational approach to more directly model the implementations of imperative PPLs.

Note that probabilistic programs with dynamic addresses and while loops pose a significant theoretical challenge, because both cases easily lead to an unbounded number of addresses as can be seen in Listing 3 and Listing 4.

```

n = sample("n", Poisson(5))
x = sample("x_" + str(n), Normal(0., 1.))

```

Listing 3. Program with random address defining an unbounded number of variables.

```

b = true; i = 0;
while b do
    i = i + 1
    b = sample("b_" + i, Bernoulli(0.25))

```

Listing 4. Program with while loop implementing a Geometric distribution.

2.2 Remark on a Measure-Theoretic Interpretation

Even though it suffices to consider the explicit computation of the density of a probabilistic program in this work, a measure-theoretic interpretation of the presented operational semantics is still worth exploring. The density of a measure μ with respect to some reference measure ν is formally given by the Radon-Nikodym derivative p such that $\mu(A) = \int_A p \, d\nu$. Thus, the measure-theoretic interpretation of the operational semantics is described by a construction of a measure space of traces $(\mathcal{T}, \Sigma_{\mathcal{T}}, \nu)$ such that the density p_S is measurable. Then, $\mu_S(A) = \int_A p_S \, d\nu$ is the measure denoted by the program S . As our results do not rely on measure theoretic assumptions, we refer to Appendix C for a sketch of this measure space.

2.3 Remark on Conditioning

In the presented semantics conditioning is only supported via soft-constraints [54]. This means that the input trace tr is assumed to be fixed to observed data at certain addresses. If those addresses are encountered during execution at sample statements the data likelihood is multiplied to $\sigma(\mathbf{p})$. Informally, the program density p_S corresponds to the unnormalised posterior density. Again, a rigorous treatment of the normalisation of p_S is not required for our results.

3 Static Factorisation for Programs without Loops

The goal of this work is to statically find a factorisation of the density implicitly defined by a probabilistic program in terms of addresses. This static analysis will operate on the control-flow graph (CFG) of a program. We begin by considering programs without loops first and describe how we can formally translate such a program to its corresponding CFG. We will equip this CFG with semantics equivalent to the operational semantics of the original program. The following construction is fairly standard but necessary for formally verifying the correctness of the factorisation.

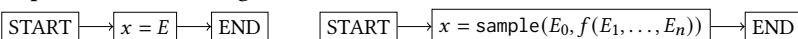
3.1 Control-Flow Graph

A CFG has five types of nodes: start, end, assign, branch, and join nodes. Branch nodes have two successor nodes, end nodes have no successor, all other nodes have one. Only join nodes have multiple predecessors. The CFG contains exactly one start node from which every other node is reachable. Further, it contains exactly one end node that is reachable from any other node. We describe the recursive translation rules for program S as diagrams, which serve to convey the general idea of the construction. For a formal description see Appendix A. Sub-graphs are drawn by circular nodes. Branch and join nodes are drawn as diamond nodes, the latter filled black. Assign, start, and end nodes have rectangular shape. In text, branch nodes are written as $\text{Branch}(E)$. Assign nodes are denoted with $\text{Assign}(x = E)$ and $\text{Assign}(x = \text{sample}(E_0, \dots))$.

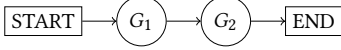
- The CFG of a skip-statement, $S = \text{skip}$, consists of only start and end nodes:



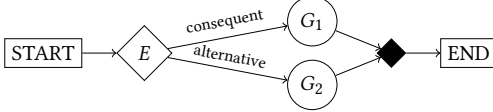
- The CFG of an assignment, $x = E$, or sample statement $x = \text{sample}(E_0, f(E_1, \dots, E_n))$, is a sequence of start, assign, and end nodes:



- The CFG for a sequence of statements, $S = S_1; S_2$, is recursively defined by "stitching together" the CFGs G_1 and G_2 of S_1 and S_2 :



- The CFG of an if statement, $S = (\text{if } E \text{ then } S_1 \text{ else } S_2)$, is also defined in terms of the CFGs of S_1 and S_2 together with a branch node $B = \text{Branch}(E)$ and a join node J :



We denote the successor nodes of the branch node B as $\text{consequent}(B)$ and $\text{alternative}(B)$. Furthermore, we say the tuple (B, J) is a branch-join pair and define $\text{BranchJoin}(G)$ to be the set of all branch-join pairs of G . Lastly, let $\text{condexp}(B)$ denote the conditional expression E .

CFG Semantics. For a control-flow graph G we define the small-step semantics for a trace tr as a relation that models the transition from node to node depending on the current program state:

$$(\sigma, N) \xrightarrow{\text{tr}} (\sigma', N')$$

The semantics for the CFG of program S are set-up precisely such that they are equivalent to the operational semantics of program S . The transition rules depending on node type and program state are given below, where we denote the unique child of non-branch nodes N in G with $\text{successor}(N)$:

$$\begin{array}{c}
 \frac{N = \text{START} \quad N' = \text{successor}(N)}{(\sigma, N) \xrightarrow{\text{tr}} (\sigma, N')} \quad \frac{N = \text{Assign}(x = E) \quad N' = \text{successor}(N)}{(\sigma, N) \xrightarrow{\text{tr}} (\sigma[x \mapsto \sigma(E)], N')} \\
 \\
 \frac{N = \text{Join} \quad N' = \text{successor}(N)}{(\sigma, N) \xrightarrow{\text{tr}} (\sigma, N')} \quad \frac{N = \text{Branch}(E) \quad \sigma(E) = \text{true} \quad N = \text{Branch}(E) \quad \sigma(E) = \text{false}}{(\sigma, N) \xrightarrow{\text{tr}} (\sigma, \text{consequent}(N))} \quad \frac{N = \text{Branch}(E) \quad \sigma(E) = \text{false}}{(\sigma, N) \xrightarrow{\text{tr}} (\sigma, \text{alternative}(N))} \\
 \\
 \frac{N = \text{Assign}(x = \text{sample}(E_0, f(E_1, \dots, E_n))) \quad N' = \text{successor}(N) \quad \forall i: \sigma(E_i) = V_i \wedge V_i \neq \text{null} \quad V_0 \in \mathbf{Strings} \quad V = \text{tr}(V_0) \wedge V \neq \text{null}}{(\sigma, N) \xrightarrow{\text{tr}} (\sigma[x \mapsto V, \mathbf{p} \mapsto \sigma(\mathbf{p}) \cdot \text{pdf}_f(V; V_1, \dots, V_n)], N')} \quad (2)
 \end{array}$$

Finally, the meaning of a CFG is a mapping from trace to density. It is defined for traces tr if there exists a path of node transitions from start to end node.

DEFINITION 2. The density of a CFG G is the function $\bar{p}_G: \mathcal{T} \times \Sigma \rightarrow \mathbb{R}_{\geq 0}$ given by

$$\bar{p}_G(\text{tr}, \sigma_0) := \begin{cases} \sigma(\mathbf{p}) & \text{if } \exists \sigma \in \Sigma, n \in \mathbb{N}: \forall i = 1, \dots, n: \exists \sigma_i, N_i: \\ & (\sigma_0, \text{START}) \xrightarrow{\text{tr}} \dots \xrightarrow{\text{tr}} (\sigma_i, N_i) \xrightarrow{\text{tr}} \dots \xrightarrow{\text{tr}} (\sigma, \text{END}) \\ \text{undefined} & \text{otherwise.} \end{cases}$$

For initial program state σ_0 as in Definition 1, we define $p_G: \mathcal{T} \rightarrow \mathbb{R}_{\geq 0}$ with $p_G(\text{tr}) = \bar{p}_G(\text{tr}, \sigma_0)$.

PROPOSITION 1. For all programs S without while loops and corresponding control-flow graph G , it holds that for all traces tr and all initial states σ_0

$$\bar{p}_S(\text{tr}, \sigma_0) = \bar{p}_G(\text{tr}, \sigma_0).$$

PROOF. By structural induction. See Appendix B.1. □

3.2 Static Provenance Analysis

The provenance of variable x in CFG node N are all the addresses $A \subseteq \text{Strings}$ that contribute to the computation of its value in node N . In this section, we define an algorithm that operates on the CFG and statically determines the provenance for any variable x at any node N . As this is a static algorithm, it is an over-approximation of the true provenance. In this section, we will prove soundness of the presented algorithm. We show that indeed the value of variable x at node N can be computed from the values in the trace tr at the addresses which make up the provenance. But first, we introduce some useful concepts which are similar to definitions found in data-flow analysis [14, 26].

DEFINITION 3. For a CFG G , let $\text{AssignNodes}(G, x)$ be all nodes N' in G that assign variable x , $N' = \text{Assign}(x = \dots)$. The set of reaching definitions for variable x in node N is defined as

$$\text{RD}(N, x) = \{N' \in \text{AssignNodes}(G, x) : \exists \text{path}(N', N_1, \dots, N_n, N), N_i \notin \text{AssignNodes}(G, x)\}.$$

Intuitively, the set of reaching definitions are all nodes that could have written the value of x in the program state before executing node N .

DEFINITION 4. For CFG G and node N the set of branch parents is defined as

$$\text{BP}(N) = \{B : (B, J) \in \text{BranchJoin}(G) \wedge \exists \text{path}(B, \dots, N, \dots, J)\}.$$

A branch node B and join node J are a branch-join pair $(B, J) \in \text{BranchJoin}(G)$ if they belong to the same if-statement (see the translation rule for if-statements to CFGs).

The set of branch parents are all parent branch nodes of N that determine if the program branch that N belongs to is executed.

Lastly, we define the set of all strings that the address expression of a sample statement may evaluate to.

DEFINITION 5. For a sample CFG node $N = \text{Assign}(x = \text{sample}(E_0, f(E_1, \dots, E_n)))$, we define the set of all possible addresses as

$$\text{addresses}(N) = \{\sigma(E_0) : \sigma \in \Sigma\} \subseteq \text{Strings}.$$

Now we are able to define the algorithm to statically approximate the provenance of any variable x at any node N denoted by $\text{prov}(N, x)$ in Algorithm 1. This algorithm is inspired by standard methods for dependence analysis. In the algorithm, we use $\text{vars}(E)$, which is defined as the set of all program variables in expression E . It is often useful to determine the provenance of an entire expression instead of only a single variable. For expressions E , we lift the definition of prov

$$\text{prov}(N, E) = \bigcup_{y \in \text{vars}(E)} \text{prov}(N, y). \quad (3)$$

The algorithm works as follows. If we want to find $\text{prov}(N, x)$, we first have to find all reaching definitions for x . If the reaching definition N' is a sample node, $x = \text{sample}(E_0, \dots)$, then the value of x can only depend on the addresses generated by E_0 , $\text{addresses}(N')$, and the provenance of E_0 . If N' is an assignment node, $x = E$, we have to recursively find the provenance of the expression E . Lastly, there can be multiple reaching definitions for x in different branches and the value of x also depends on the branching condition. Thus, we also find all branch parents and recursively determine their provenance and add it to the final result.

In practice the potentially infinite set $\text{addresses}(N)$ can be represented by the CFG node N itself. As remarked on line 8, an implementation of Algorithm 1 would return a set of CFG sample nodes rather than a potentially infinite set of strings. We call this version of the algorithm provnode and we have $\text{prov}(N, x) = \bigcup_{N' \in \text{provnode}(N, x)} \text{addresses}(N')$.

Algorithm 1 Computing the provenance set $\text{prov}(N, x)$ statically from the CFG.

```

1: Input CFG node  $N$ , variable  $x$ 
2:  $\text{result} \leftarrow \emptyset$ 
3:  $\text{queue} \leftarrow [(N, x)]$ 
4: while  $\text{queue}$  is not empty do
5:    $(N, x) \leftarrow \text{queue.pop}()$ 
6:   for  $N' \in \text{RD}(N, x)$  do
7:     if  $N' = \text{Assign}(x = \text{sample}(E_0, f(E_1, \dots, E_n)))$  then
8:        $\text{result} \leftarrow \text{result} \cup \text{addresses}(N')$   $\triangleright$  in practice:  $\text{result} \leftarrow \text{result} \cup \{N'\}$ 
9:        $E' \leftarrow E_0$   $\triangleright E_0$  comes from pattern-matching  $N'$ 
10:    else if  $N' = \text{Assign}(x = E)$  then
11:       $E' \leftarrow E$   $\triangleright E$  comes from pattern-matching  $N'$ 
12:    for  $y \in \text{vars}(E')$  do
13:      if  $\text{is\_unmarked}((N', y))$  then
14:         $\text{mark}((N', y))$ 
15:         $\text{queue.push}((N', y))$ 
16:    for  $N_{\text{bp}} \in \text{BP}(N')$   $\triangleright N_{\text{bp}} = \text{Branch}(E_{N_{\text{bp}}})$  do
17:      for  $y \in \text{vars}(E_{N_{\text{bp}}})$  do
18:        if  $\text{is\_unmarked}((N_{\text{bp}}, y))$  then
19:           $\text{mark}((N_{\text{bp}}, y))$ 
20:           $\text{queue.push}((N_{\text{bp}}, y))$ 
21: return  $\text{result}$ 

```

We get a clearer sense of the introduced definitions and Algorithm 1 by considering a simple program in Figure 1. The right panel shows the CFG with reaching definitions and branch parents for several nodes drawn as arrows. These arrows illustrate how Algorithm 1 computes the provenance of variables m and s at the sample node for address "x". Note that if we were to change the if statement to `(if (b == 1) then (m = 1) else (m = 1))`, then the address "b" would still be in the provenance set of m . This illustrates that the presented algorithm can only over-approximate the provenance and not compute it exactly.

```

b = sample("b", Bernoulli(0.5))
s = sample("s", InverseGamma(1., 1.))
if b == 1 then
  m = sample("mu", Normal(0., 1.))
else
  m = 1
x = sample("x", Normal(m, s))

```

$N_7 = \text{Assign}(x = \dots)$, $V_s^{N_7}(\text{tr}) = \text{tr}("s")$,

$$V_m^{N_7}(\text{tr}) = \begin{cases} \text{tr}("mu") & \text{if } \text{tr}("b") = \text{true}, \\ 1 & \text{otherwise.} \end{cases}$$

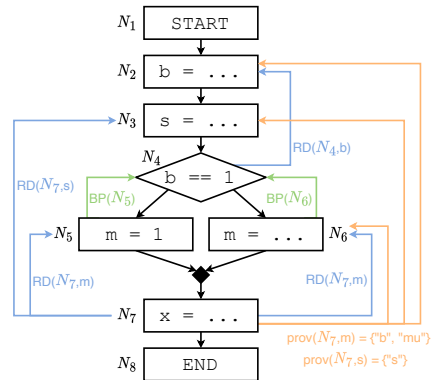


Fig. 1. Reaching definitions, branch parents, and the provenance set for a simple example. The evaluation functions introduced in Section 3.2 are also given for node N_7 .

Soundness. Recall that the purpose of finding the provenance of variable x at node N is to determine which addresses contribute to the computation of its value. To prove that Algorithm 1 indeed finds a sufficient set of addresses, first, we have to precisely define what we mean that a set of addresses contribute to the computation. Suppose that there is a function f that maps a trace tr to a value v . The set of addresses A contribute to the computation of value v , if any other trace tr' that has the same values as tr for all addresses $\alpha \in A$, is mapped to the same value $f(\text{tr}') = v$. In other words, if we change the trace tr at an address $\beta \in \text{Strings} \setminus A$, the value of $f(\text{tr})$ remains unchanged.

DEFINITION 6. For a function from traces to an arbitrary set B , $f : \mathcal{T} \rightarrow B$, we write $f \in [\mathcal{T}|_A \rightarrow B]$ for a subset of addresses $A \subseteq \text{Strings}$, if and only if

$$\forall \alpha \in A: \text{tr}(\alpha) = \text{tr}'(\alpha) \implies f(\text{tr}) = f(\text{tr}').$$

That is, changing the values of tr at addresses $\beta \in \text{Strings} \setminus A$ does not change the value $f(\text{tr})$.

The next lemma states that we can always over-approximate the set of addresses that contribute to the computation of the values $f(\text{tr})$.

LEMMA 1. If $A \subseteq A'$ and $f \in [\mathcal{T}|_A \rightarrow B]$, then $f \in [\mathcal{T}|_{A'} \rightarrow B]$. In other words,

$$A \subseteq A' \implies [\mathcal{T}|_A \rightarrow B] \subseteq [\mathcal{T}|_{A'} \rightarrow B]. \quad (4)$$

If we combine two functions with provenance A_1 and A_2 respectively, then the resulting function has provenance $A_1 \cup A_2$. This is shown in the following lemma.

LEMMA 2. Let $f_1 \in [\mathcal{T}|_{A_1} \rightarrow B_1]$, $f_2 \in [\mathcal{T}|_{A_2} \rightarrow B_2]$, $h: B_1 \times B_2 \rightarrow C$. Then

$$\text{tr} \mapsto h(f_1(\text{tr}), f_2(\text{tr})) \in [\mathcal{T}|_{A_1 \cup A_2} \rightarrow C].$$

This brings us to the main soundness result of Algorithm 1. Proposition 2 states that we can equip each CFG node N with evaluation functions that depend on the addresses determined by prov in the sense of Definition 6. The evaluation functions directly map the trace tr to the values of variables at node N such that the values agree with the small-step CFG semantics relation $\xrightarrow{\text{tr}}$. Importantly, while the operational semantics are defined for each individual trace, the evaluation functions work for all traces. In Fig. 1, concrete examples of these evaluation functions for a simple program can be found.

PROPOSITION 2. Let G be the CFG of a program S without while loop statements. For each node N and variable x , there exists an evaluation function

$$V_x^N \in [\mathcal{T}|_{\text{prov}(N,x)} \rightarrow \mathcal{V}],$$

such that for all traces tr , initial program state σ_0 as in Definition 1, and execution sequence

$$(\sigma_0, \text{START}) \xrightarrow{\text{tr}} \dots \xrightarrow{\text{tr}} (\sigma_i, N_i) \xrightarrow{\text{tr}} \dots \xrightarrow{\text{tr}} (\sigma_l, \text{END})$$

we have

$$\sigma_i(x) = V_x^{N_i}(\text{tr}).$$

PROOF. The CFG G of a program without while loops is a directed acyclic graph and the statement is proven by induction on G . See Appendix B.2. $\square$

Intuitively, V_x^N computes the value of x before executing N . These evaluation functions V_x^N can be lifted to evaluation functions for expressions V_E^N :

$$V_c^N(\text{tr}) = c, \quad V_{g(E_1, \dots, E_n)}^N(\text{tr}) = g(V_{E_1}^N(\text{tr}), \dots, V_{E_n}^N(\text{tr})).$$

By Lemma 2, if $V_y^N \in [\mathcal{T}|_{\text{prov}(N,y)} \rightarrow \mathcal{V}]$ for all $y \in \text{vars}(E)$, then $V_E^N \in [\mathcal{T}|_{\text{prov}(N,E)} \rightarrow \mathcal{V}]$.

3.3 Static Factorisation Theorem for Programs without Loops

With Proposition 2, we can prove the first factorisation theorem by combining the evaluation functions V_x^N to express the program density p in a factorised form.

THEOREM 1. *Let G be the CFG for a program S without while loop statements. Let $N_1, \dots, N_K$ be all sample nodes in G . For each sample node $N_k = \text{Assign}(x_k = \text{sample}(E_0^k, f^k(E_1^k, \dots, E_{n_k}^k)))$, let*

$$A_k = \text{addresses}(N_k) \cup \bigcup_{i=0}^{n_k} \text{prov}(N_k, E_i^k) \cup \bigcup_{N' \in \text{BP}(N_k)} \text{prov}(N', \text{conexp}(N')).$$

Then, there exist functions $p_k \in [\mathcal{T}|_{A_k} \rightarrow \mathbb{R}_{\geq 0}]$ such that for all $\text{tr} \in \mathcal{T}$, if $p_S(\text{tr}) \neq \text{undefined}$, then

$$p_S(\text{tr}) = p_G(\text{tr}) = \prod_{k=1}^K p_k(\text{tr}).$$

PROOF. For each sample node N_k , define $b_k(\text{tr}) := \bigwedge_{N' \in \text{BP}(N_k)} t_{N'}^k(V_{\text{conexp}(N')}^{N'}(\text{tr}))$ where $t_{N'}^k(v) = v$ if N_k is in the consequent branch of N' and $t_{N'}^k(v) = \neg v$ if N_k is in the alternate branch. By Lemma 2, we have $b_k \in [\mathcal{T}|_{\bigcup_{N' \in \text{BP}(N_k)} \text{prov}(N', \text{conexp}(N'))} \rightarrow \{\text{true}, \text{false}\}]$.

Above, $V_{\text{conexp}(N')}^{N'}(\text{tr})$ are precisely the evaluations of the branch conditions for node N_k . Thus, $b_k(\text{tr})$ is true if N_k is in the execution sequence for tr else false. Define the factor p_k as

$$p_k(\text{tr}) := \delta_{b_k(\text{tr})} \text{pdf}_{f^k} \left(\text{tr}(V_{E_0^k}^{N_k}(\text{tr})); V_{E_1^k}^{N_k}(\text{tr}), \dots, V_{E_{n_k}^k}^{N_k}(\text{tr}) \right) + (1 - \delta_{b_k(\text{tr})}). \quad (5)$$

By Proposition 2, $V_{E_i^k}^{N_k} \in [\mathcal{T}|_{\text{prov}(N_k, E_i^k)} \rightarrow \mathcal{V}]$ and $V_{E_0^k}^{N_k} \in [\mathcal{T}|_{\text{prov}(N_k, E_0^k)} \rightarrow \text{addresses}(N_k)]$. Thus, it follows from Lemma 2 that $p_k \in [\mathcal{T}|_{A_k} \rightarrow \mathbb{R}_{\geq 0}]$ by construction. Lastly, from Proposition 2 we know that for the end-state $\sigma_l(\mathbf{p}) = V_{\mathbf{p}}^{\text{END}}(\text{tr}) = p_G(\text{tr})$. Since $b_k(\text{tr}) = \text{true}$ if and only if N_k is in the execution sequence for tr and since the $\text{pdf}_{f^k}(\dots)$ term in (5) is precisely the value that is multiplied to $\mathbf{p}$ at sample statements (1), we have $p_G(\text{tr}) = \prod_{k=1}^K p_k(\text{tr})$. $\square$

Again, consider the example program in Fig. 1. The density of this model factorises like below:

$$\begin{aligned} p(\text{tr}) &= \text{pdf}_{\text{Bernoulli}}(\text{tr}(\text{"b"}); 0.5) && \in [\mathcal{T}|_{\{\text{"b"}\}} \rightarrow \mathbb{R}_{\geq 0}] \\ &\cdot \text{pdf}_{\text{InverseGamma}}(\text{tr}(\text{"s"}); 1, 1) && \in [\mathcal{T}|_{\{\text{"s"}\}} \rightarrow \mathbb{R}_{\geq 0}] \\ &\cdot (\delta_{\text{tr}(\text{"b"})} \text{pdf}_{\text{Normal}}(\text{tr}(\text{"mu"}); 0, 1) + (1 - \delta_{\text{tr}(\text{"b"})})) && \in [\mathcal{T}|_{\{\text{"b"}, \text{"mu"}\}} \rightarrow \mathbb{R}_{\geq 0}] \\ &\cdot \text{pdf}_{\text{Normal}}(\text{tr}(\text{"x"}); V_m^{N_7}(\text{tr}), V_s^{N_7}(\text{tr})) && \in [\mathcal{T}|_{\{\text{"x"}, \text{"b"}, \text{"mu"}, \text{"s"}\}} \rightarrow \mathbb{R}_{\geq 0}] \end{aligned}$$

This is exactly the factorisation obtained from Theorem 1. However, if we would replace the if-statement with a nonsensical one, (if (b == 1) then (m = 1) else (m = 1)), then the fourth factor is in $[\mathcal{T}|_{\{\text{"x"}, \text{"b"}, \text{"s"}\}} \rightarrow \mathbb{R}_{\geq 0}]$ with $V_m^{N_7}(\text{tr}) = \text{ife}(\text{tr}(\text{"b"}), 1, 1) = 1$, where we use ife as a short-hand for the if-else case distinction. In this case, the static analysis over-approximates the true provenance with the spurious dependency on "b".

3.3.1 Equivalence to Bayesian Networks. If we restrict the way sample statements are used in our PPL, we can establish a known equivalence to Bayesian networks. A Bayesian network [31] is a directed acyclic graph $\mathcal{G}$ over a finite set of nodes $X_1, \dots, X_n$ which represent random variables, such that their joint distribution factorises according to the graph $\mathcal{G}$,

$$P(X_1, \dots, X_n) = \prod_{i=1}^n P(X_i \mid \text{parents}_{\mathcal{G}}(X_i)).$$

If we assume that all sample statements of a program have a constant unique address α_k , $N_k = \text{Assign}(x_k = \text{sample}(\alpha_k, f^k(E_1^k, \dots, E_{n_k}^k)))$, then we can identify each factor $p_k \in [\mathcal{T}|_{A_k} \rightarrow \mathbb{R}_{\geq 0}]$

of Theorem 1 with the unique address $\alpha_k \in \text{Strings}$. We construct the Bayesian network $\mathcal{G}$ by mapping each address to a random variable X_{α_k} . Since α_k is unique, p_k is the only relevant factor for X_{α_k} and can be interpreted as conditional density function of X_{α_k} . To see this, rewrite

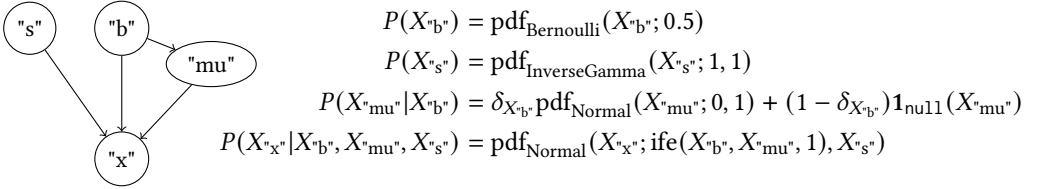
$$p_k(\text{tr}) = \delta_{b_k(\text{tr})} \text{pdf}_{f^k} \left(\text{tr}(\alpha_k); V_{E_1^k}^{N_k}(\text{tr}), \dots, V_{E_{n_k}^k}^{N_k}(\text{tr}) \right) + (1 - \delta_{b_k(\text{tr})}) \mathbf{1}_{\text{null}}(\text{tr}(\alpha_k)),$$

where the function $\mathbf{1}_{\text{null}}(v)$ equals 1 if $v = \text{null}$ else 0. If the node N_k is not executed, then the value of X_{α_k} is assumed to be null and we add the $\mathbf{1}_{\text{null}}(\text{tr}(\alpha_k))$ term to p_k . As pdf_{f^k} is a probability density function and $\mathbf{1}_{\text{null}}$ is the density of a Dirac distribution centered at null, the factor p_k is also a density function for each choice of values in tr at addresses $\alpha \in A_k \setminus \{\alpha_k\}$. Thus, we introduce the edge $\alpha_j \rightarrow \alpha_k$ if $\alpha_j \in A_k \setminus \{\alpha_k\}$, such that $\text{parents}_{\mathcal{G}}(X_{\alpha_k}) = A_k \setminus \{\alpha_k\}$ and

$$P(X_{\alpha_k} | \text{parents}_{\mathcal{G}}(X_{\alpha_k})) = p_k(\{\alpha \mapsto X_{\alpha} : \alpha \in A_k\}).$$

In the above construction, it is important that each sample statement has a *unique* address. This guarantees no cyclic dependencies and a well-defined conditional probability distributions.

The Bayesian network equivalent to the example program of Fig. 1 is drawn below. Since the value of the program variable m depends on which program branch is taken during execution, the random variable $X_{\text{"x"}}$ not only depends on $X_{\text{"mu"}}$, but also on $X_{\text{"b"}}$.



3.3.2 Equivalence to Markov Networks. If the sample addresses are not unique or not constant, then the factorisation is in general not a Bayesian network. However, we can show equivalence to Markov networks. A Markov network [31] is an *undirected* graph $\mathcal{H}$ over random variables $\vec{X}$ that represents their dependencies. We consider Markov networks where the joint distribution of $\vec{X}$ factorises over a set of cliques $\mathcal{D} = \{D_1, \dots, D_n\} \subseteq \text{cliques}(\mathcal{H})$, with $\bigcup_{i=1}^n D_i = \vec{X}$:

$$P(\vec{X}) = \prod_{i=1}^n \phi_i(D_i).$$

We again identify a random variable X_{α} for each address $\alpha \in \bigcup_{k=1}^K A_k$ and construct a Markov network $\mathcal{H}$ by connecting node X_{α} to X_{β} if $\alpha \in A_k \wedge \beta \in A_k$ for any k . Thus, $D_k = \{X_{\alpha} : \alpha \in A_k\}$ forms a clique and p_S factorises over $\mathcal{H}$ with $\phi_k(D_k) = p_k(\{\alpha \mapsto X_{\alpha} : \alpha \in A_k\})$.

Consider the program with constant but non-unique sample addresses in Listing 5. With nine sample statements we get a Markov network over $\{X_{\text{"F"}}, X_{\text{"P0"}}, X_{\text{"P1"}}, X_{\text{"D0"}}, X_{\text{"D1"}}\}$. The nine factors correspond to the address sets $\{\text{"F"}\}$, $\{\text{"F"}, \text{"P0"}\}$, $\{\text{"F"}, \text{"P1"}\}$, $\{\text{"F"}, \text{"D0"}, \text{"P1"}\}$, $\{\text{"F"}, \text{"D1"}, \text{"P0"}\}$, $2 \times \{\text{"F"}, \text{"P0"}, \text{"D0"}\}$, and $2 \times \{\text{"F"}, \text{"P1"}, \text{"D1"}\}$. In the consequent branch of the program, the distribution in the sample statement with address "P1" depends on "D0", while in the alternative branch, the distribution of "P0" depends on "D1". Thus, there is the dependency cycle "P0" $\rightarrow$ "D0" $\rightarrow$ "P1" $\rightarrow$ "D1" $\rightarrow$ "P0", which means the obtained factorisation is not a Bayesian network representation for the program. Note that the program can also be rewritten with dynamic addresses to eliminate the if statement. In this case, the factorisation still admits cyclic dependencies.

Lastly, while the programs considered up until this point were equivalent to finite Bayesian networks or finite Markov networks, note that the small program in Listing 3 is equivalent to a Markov network with an infinite number of nodes $\{X_{\text{"n"}}\} \cup \{X_{\text{"x_i"}} : i \in \mathbb{N}\}$.

```

first_city_ixs = sample("F", Bernoulli(0.5))
if first_city_ixs == 0 then
  prep_0 = sample("P0", Bernoulli(0.5))
  damage_0 = sample("D0", Bernoulli(prepare_0 == 1 ? 0.20 : 0.80))
  prep_1 = sample("P1", Bernoulli(damage_0 == 1 ? 0.75 : 0.50))
  damage_1 = sample("D1", Bernoulli(prepare_1 == 1 ? 0.20 : 0.80))
else
  prep_1 = sample("P1", Bernoulli(0.5))
  damage_1 = sample("D1", Bernoulli(prepare_1 == 1 ? 0.20 : 0.80))
  prep_0 = sample("P0", Bernoulli(damage_1 == 1 ? 0.75 : 0.50))
  damage_0 = sample("D0", Bernoulli(prepare_0 == 1 ? 0.20 : 0.80))

```

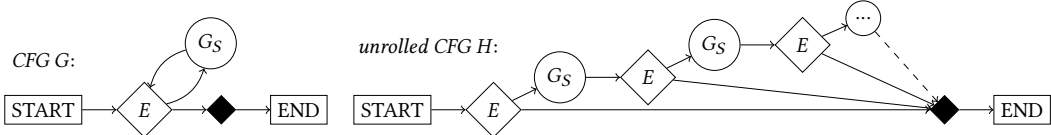
Listing 5. Probabilistic program for the hurricane example of [42].

4 Static Factorisation for Programs with Loops

In this section, we describe how the argument for proving the factorisation theorem for programs without loops, can be extended to generalise the result to programs with loops.

4.1 Unrolled Control-Flow Graph

The translation rules for programs to their CFG can be extended to support while loops as follows. Let G_S be the sub-CFG of statement S . The CFG of (while E do S) is given below on the left:



For this CFG, $\text{prov}(N, x)$ is still well-defined by Algorithm 1, but the proof of Proposition 2 does not work anymore since it requires a directed acyclic graph.

To establish Proposition 2 for programs with while loops, we introduce a second type of control-flow graph, the *unrolled CFG*, which again is a directed acyclic graph. All translation rules are as before except for while loops, which is given below and illustrated above on the right.

The resulting graph G contains for every $i \in \mathbb{N}$ branch nodes $B_i = \text{Branch}(E)$ and *copies* of the sub-CFG G_i of statement S . It contains a single join node J for which we have $(B_i, J) \in \text{BranchJoin}(G)$ for all i . Let N_{first}^i be successor of the start node in G_i and N_{last}^i the predecessor of the end node in G_i . In the new graph G , the first branch node B_1 is successor of the start node and the end node is the successor of the join node J . The branch nodes B_i have two successors N_{first}^i and J . N_{last}^i , the last node of sub-graph G_i , resumes in the next branch node B_{i+1} .

This definition implies that the number of nodes in G is countable. Furthermore, since the unrolled CFG consists only of start, end, branch, join, and assign nodes, we can equip it with the same semantics as the standard CFG. As before, the semantics are equivalent.

PROPOSITION 3. *For all program S with CFG G and unrolled CFG H , it holds that for all traces tr and initial states σ_0*

$$\bar{p}_S(\text{tr}, \sigma_0) = \bar{p}_G(\text{tr}, \sigma_0) = \bar{p}_H(\text{tr}, \sigma_0).$$

PROOF. By structural induction. See Appendix B.3.

As mentioned, Algorithm 1 still works for the unrolled CFG. However, since the unrolled CFG contains an infinite number of nodes, in practice, we run this algorithm on the standard CFG with a finite number of nodes avoiding loops by marking visited nodes (line 14 and 19). We will show

that the computations on the standard CFG produce an over-approximation of the provenance set in the unrolled CFG. But first, we define the connection between the two graph types.

DEFINITION 7. Let G be the CFG and H the unrolled CFG of program S . For each node $M \in H$, we define $\iota_{\text{cfg}}(M) \in G$ as the node from which M was copied in the construction of H .

The definition of ι_{cfg} is best understood when considering the example in Fig. 2, where the CFG and unrolled CFG of a program are depicted side-by-side and connected with the mapping ι_{cfg} .

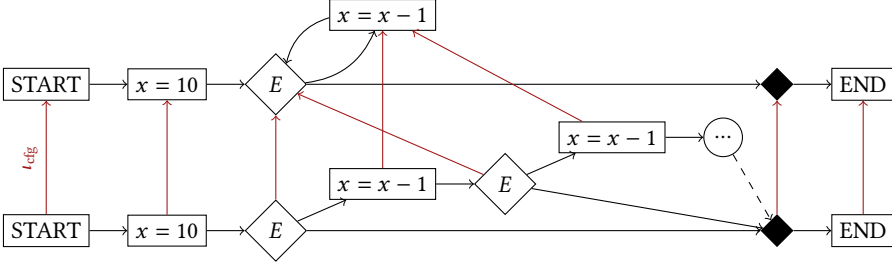


Fig. 2. CFG and unrolled CFG for program $(x = 10; \text{ while } (x < 10) \text{ do } (x = x - 1))$ and their connection via the map ι_{cfg} shown with red edges.

The following lemma states that we can over-approximate the provenance set for variable x at node M in the unrolled CFG H by applying Algorithm 1 to the corresponding node $\iota_{\text{cfg}}(M)$ in G .

LEMMA 3. Let G be the CFG and H the unrolled CFG of program S . For each node $M \in H$ following statements hold:

$$\{\iota_{\text{cfg}}(M') : M' \in \text{RD}(M, x)\} \subseteq \text{RD}(\iota_{\text{cfg}}(M), x) \quad (6)$$

$$\{\iota_{\text{cfg}}(M') : M' \in \text{BP}(M)\} = \text{BP}(\iota_{\text{cfg}}(M)) \quad (7)$$

$$\text{prov}(M, x) \subseteq \text{prov}(\iota_{\text{cfg}}(M), x) \quad (8)$$

PROOF. See Appendix B.4. □

4.2 Static Factorisation Theorem for Programs with Loops

The unrolled CFG has the important property that it is a directed acyclic graph with a potentially infinite number of nodes, but with a single root node. Further, each node $N \neq \text{START}$ still has exactly one predecessor except for join nodes. Therefore, Proposition 2 can be generalised to unrolled CFGs with only a slight modification to the proof.

PROPOSITION 4. Let H be the unrolled CFG of a program S . For each node M and variable x , there exists an evaluation function

$$V_x^M \in [\mathcal{T}|_{\text{prov}(M, x)} \rightarrow \mathcal{V}],$$

such that for all traces tr , initial program state σ_0 as in Definition 1, and execution sequence

$$(\sigma_0, \text{START}) \xrightarrow{\text{tr}} \dots \xrightarrow{\text{tr}} (\sigma_i, M_i) \xrightarrow{\text{tr}} \dots \xrightarrow{\text{tr}} (\sigma_l, \text{END})$$

we have

$$\sigma_i(x) = V_x^{M_i}(\text{tr}).$$

PROOF. The unrolled CFG H is constructed in a way such that we can prove the statement by well-founded induction on the relation $M_1 \prec M_2 \Leftrightarrow M_1$ is parent of M_2 (i.e. there is an edge from M_1 to M_2 in H) similar to the proof of Proposition 2. The full proof is given in Appendix B.5. □

As before, Proposition 4 allows us to explicitly construct the factorisation of the program density.

THEOREM 2. *Let G be the CFG for a program S . Let $N_1, \dots, N_K$ be all sample nodes in G . For each sample node $N_k = \text{Assign}(x_k = \text{sample}(E_0^k, f^k(E_1^k, \dots, E_{n_k}^k)))$, let*

$$[A_k = \text{addresses}(N_k) \cup \bigcup_{i=0}^{n_k} \text{prov}(N_k, E_i^k) \cup \bigcup_{N' \in \text{BP}(N_k)} \text{prov}(N', \text{condexp}(N'))]. \quad (9)$$

Then, there exist functions $p_k \in [\mathcal{T}|_{A_k} \rightarrow \mathbb{R}_{\geq 0}]$ such that for all $\text{tr} \in \mathcal{T}$, if $p_S(\text{tr}) \neq \text{undefined}$, then

$$p_S(\text{tr}) = p_G(\text{tr}) = p_H(\text{tr}) = \prod_{k=1}^K p_k(\text{tr}).$$

PROOF. We give a proof sketch and refer to Appendix B.6 for the full proof. For each sample node M_j in the *unrolled* CFG H , we construct $\tilde{p}_j \in [\mathcal{T}|_{\tilde{A}_j} \rightarrow \mathbb{R}_{\geq 0}]$ as in the proof of Theorem 1. Next, we group the nodes M_j by their corresponding CFG node: $p_k = \prod_{j: \iota_{\text{cfg}}(M_j) = N_k} \tilde{p}_j$. From Lemma 3, we have that if $\iota_{\text{cfg}}(M_j) = N_k$, then $\tilde{A}_j \subseteq A_k$. Thus, $p_k \in [\mathcal{T}|_{A_k} \rightarrow \mathbb{R}_{\geq 0}]$. Like in the proof of Theorem 1 it follows from Proposition 4 that $p_S(\text{tr}) = p_G(\text{tr}) = \prod_{k=1}^K p_k(\text{tr})$. $\square$

Equivalence to Markov Networks. As in Section 3.3.2, Theorem 2 implies that a program S with K sample statements is equivalent to a Markov network $\mathcal{H}$ with nodes $\{X_\alpha: \alpha \in \bigcup_{k=1}^K A_k\}$ even if it contains loops. Again, the sets $D_k = \{X_\alpha: \alpha \in A_k\}$ form cliques in $\mathcal{H}$ and p_S factorises over $\mathcal{H}$ with $\phi_k(D_k) = p_k(\{\alpha \mapsto X_\alpha: \alpha \in A_k\})$: $P(\vec{X}) = \prod_{k=1}^K \phi_k(D_k)$.

5 Optimising Posterior Inference Algorithms

While the statically obtained factorisation is provably correct, it is natural to ask whether the quality of the factorisation is good enough to be of practical use. To answer this question, we have developed a program transformation based on the static factorisation that generates sub-programs for each factor (described in Section 5.1). This program transformation operates on probabilistic programs that are implemented in our research PPL which models the formal language introduced in Section 2. It is implemented in Julia and can be viewed as a subset of PPLs like Gen, Turing, or Pyro. Many posterior inference algorithms can be improved by exploiting the factorisation of the model density. In Section 5.2, we demonstrate that if the change to the current program trace is small as in the light-weight Metropolis Hastings kernel [58], then we can leverage the sub-programs to compute the acceptance probability faster. In Section 5.3, we show that we can reduce the variance of the ELBO gradient estimation in variational inference. Lastly, in Section 5.4, we compile sub-programs to enable an iterative implementation of sequential Monte Carlo. The realised optimisation tricks are well-known but typically restricted to programs with a finite number of random variables, PPLs that require the user to explicitly construct the graphical structure, or dynamic approaches. In contrast, our novel transformation operates on the source code and makes the optimisations statically available for programs containing while loops and dynamic addresses. We evaluate our implementation [10] on a benchmark set of 16 probabilistic programs, the majority of which make use of while loops to declare an unbounded number of random variables.

5.1 Generating Sub-programs from the Static Factorisation

We generate a sub-program for each sample statement (and thus for each factor) by deriving a program slicing method from the static provenance analysis. We want to generate a sub-program for each sample node $N_k = \text{Assign}(x_k = \text{sample}(E_0^k, f^k(E_1^k, \dots, E_{n_k}^k)))$ that

- (1) enables us to continue execution directly from N_k from any state σ ,
- (2) stops execution once all dependencies of N_k were reached,
- (3) supports multiple inference algorithms.

To realise (1) and (2), we collect all sample nodes that depend on N_k . From Theorem 2 we know that a sample node N_j depends on N_k if N_k contributes to the provenance A_j of the factor $p_j \in [\mathcal{T}|_{A_j} \rightarrow \mathbb{R}_{\geq 0}]$ as defined in Eq. (9). As explained in Section 3.2, these nodes can be found by modifying Algorithm 1 on line 8 to collect sample nodes instead of their addresses. For this modified algorithm `provnode`, we have $\text{prov}(N, x) = \bigcup_{N' \in \text{provnode}(N, x)} \text{addresses}(N')$ and N_j depends on N_k if any variable y in any expression E_i^j in the sample statement of N_j or in the conditional expression of any branch parent of N_j depends on N_k . We write $j \in \mathcal{J}_k$ if N_j depends on N_k .

We slice the program to only retain nodes that lie on a path from N_k to N_j for some $j \in \mathcal{J}_k$. More formally, let G_k be the CFG that contains node N if there exists $j \in \mathcal{J}_k$ such that there is a path from N_k to N_j that visits N and does not revisit N_k . The start node of G_k has successor N_k and the nodes N_j , $j \in \mathcal{J}_k$, are connected to the end node of G_k if they have no successor in G_k . Lastly, sample nodes N_j in G_k that do not depend on N_k , $j \notin \mathcal{J}_k$, are transformed to *read* nodes for which $(\sigma, x = \text{read}(E_0)) \Downarrow^{\text{tr}} \sigma[x \mapsto \text{tr}(\sigma(E_0))]$. This construction is made precise in Appendix D.1, where we also prove following correctness theorems as consequences of Theorem 2.

THEOREM 3. *Let tr_1 and tr_2 be traces that only differ at address α , i.e. $\text{tr}_1(\alpha) \neq \text{tr}_2(\alpha)$ and $\forall \beta \neq \alpha: \text{tr}_1(\beta) = \text{tr}_2(\beta)$. Let the execution sequences in the unrolled CFG H be equal up to sample node $M_\alpha = \text{Assign}(x = \text{sample}(E_0, f(E_1, \dots, E_n)))$ and state σ , such that $\alpha = \sigma(E_0)$:*

$$\begin{aligned} (\sigma_0, \text{START}) &\xrightarrow{\text{tr}_1} \dots \xrightarrow{\text{tr}_1} (\sigma, M_\alpha) \xrightarrow{\text{tr}_1} \dots \xrightarrow{\text{tr}_1} (\sigma_{l_1}, \text{END}), \\ (\sigma_0, \text{START}) &\xrightarrow{\text{tr}_2} \dots \xrightarrow{\text{tr}_2} (\sigma, M_\alpha) \xrightarrow{\text{tr}_2} \dots \xrightarrow{\text{tr}_2} (\sigma_{l_2}, \text{END}), \end{aligned}$$

If M_α is the only sample node in the execution sequences whose address expression evaluates to α , then for the sliced CFG G_k of sample node $N_k = \iota_{\text{cfg}}(M_\alpha)$, it holds that

$$\log p_G(\text{tr}_1) - \log p_G(\text{tr}_2) = \log \bar{p}_{G_k}(\text{tr}_1, \sigma) - \log \bar{p}_{G_k}(\text{tr}_2, \sigma).^1$$

PROOF. Follows from Theorem 2 and two technical lemmas. See Appendix D.2. $\square$

COROLLARY 1. *Assume that for each trace tr there is at most one sample node M_α in its execution sequences such that the address expression of M_α evaluates to α . For initial program state σ_0 as in Definition 1, define*

$$p_\alpha(\text{tr}) := \begin{cases} \bar{p}_{G_k}(\text{tr}, \sigma) & \text{if } \exists \sigma \in \Sigma, M_\alpha: (\sigma_0, \text{START}) \xrightarrow{\text{tr}} \dots \xrightarrow{\text{tr}} (\sigma, M_\alpha) \wedge \iota_{\text{cfg}}(M_\alpha) = N_k \\ 1 & \text{otherwise.} \end{cases}$$

Then, $\Delta_\alpha(\text{tr}) := \log p_G(\text{tr}) - \log p_\alpha(\text{tr})$ is independent of address α , $\Delta_\alpha \in [\mathcal{T}|\text{Strings} \setminus \{\alpha\} \rightarrow \mathbb{R}_{\geq 0}]$.

PROOF. Follows directly from Theorem 3. See Appendix D.3. $\square$

Lastly, to fulfill (3), we further transform the program to take as input an *execution context* ctx . The context controls the behavior when executing a sample statement, e.g. to resample a variable or to accumulate the density. To actually handle the effects of different inference algorithms, we replace sample statements in the original program with three abstract methods that delegate the behavior to the context ctx . In G_k , the statement of N_k is transformed to *visit*, the statements of the nodes that depend on N_k to *score*, and all other sample nodes that are on a path between N_k and some N_j , $j \in \mathcal{J}_k$, but do not depend on N_k , are transformed to *read*. Concrete instances of such contexts will be presented in the following sections and formalised in Appendix D.

We have implemented the sub-program generation as a compilation to the Julia programming language making use of multiple dispatch for contextualised execution. To enable continuation

¹As most PPLs perform computations in terms of log-densities, we will often write $\log a - \log b$ instead of a/b .

from any program state σ we also inject a mutable state struct ρ . As the CFG slicing implementation is a fairly standard graph traversal algorithm, we refer to our implementation [10] for further details like the translation from CFGs to source code.

The sub-program generation process is best illustrated with three examples. Consider Program 1 in Figure 3. To generate a sub-program for the factor of sample statement "B", we omit the sample statements "A" and "E" as they do not lie on a path from "B" to "D" — the only sample statement that depends on "B". However, we still need to read the value of program variable A from the cached program state ρ and we read the value for random variable "C" from the trace. Continuing execution directly from a sample statement by caching the program state means that we can continue a loop from a specific iteration as exemplified by Program 2 in Figure 3. This is particularly powerful, if the only dependency of a sample statement belongs to the same loop iteration. In this case, an entire while loop can be replaced by a single iteration which is shown for Program 3 in Figure 3. Note that this form of loop-iteration independence is not captured by the factorisation of Proposition 4. Instead, it is product of the program slicing implementation.

Original formal program	Julia sub-program for factor (input: ctx and ρ)
<pre># Program 1 A = sample("A", Norm(0.0, 1.0)) B = sample("B", Norm(A, 1.0)) C = sample("C", Norm(A, 1.0)) D = sample("D", Norm(B+C, 1.0)) E = sample("E", Norm(A, 1.0))</pre>	<pre># sub-program for factor of variable "B" ρ.B = visit(ctx, ρ, "B", Norm(ρ.A, 1.0)) ρ.C = read(ctx, ρ, "C") ρ.D = score(ctx, ρ, "D", Norm(ρ.B+ρ.C, 1.0))</pre>
<pre># Program 2 i = 0; b = 1; while b do b = sample("b"+str(i), Bern(0.5)) i = i + 1</pre>	<pre># sub-program for factor of variable "b" ρ.b = visit(ctx, ρ, "b"*str(ρ.i), Bern(0.5)) ρ.i = ρ.i + 1 while b ρ.b = score(ctx, ρ, "b"*str(ρ.i), Bern(0.5)) ρ.i = ρ.i + 1 end</pre>
<pre># Program 3 i = 0; while i < N do z = sample("z"+str(i), Bern(0.5)) m = (z == 1) ? -2.0 : 2.0 x = sample("x"+str(i), Norm(m, 1.0)) i = i + 1</pre>	<pre># sub-program for factor of variable "z" ρ.z = visit(ctx, ρ, "z"*str(ρ.i), Bern(0.5)) ρ.m = (ρ.z == 1) ? -2.0 : 2.0 ρ.x = score(ctx, ρ, "x"*str(ρ.i), Norm(ρ.m, 1.0))</pre>

Fig. 3. Example Julia sub-programs generated from the static factorisation (with short-hands Norm = Normal, Bern = Bernoulli, str = string). Program variables are read from and stored to the program state struct ρ .

5.2 Speeding-up Acceptance Probability Computation for Single-Site Trace Updates

In this section we introduce execution contexts to speed-up the single-site or light-weight Metropolis Hastings inference (LMH) algorithm [58]. The LMH algorithm works by selecting a single address α in each iteration at random and proposing a new value v for the current trace $\text{tr}(\alpha)$. Even though we select a single address for the update, the proposal may have to produce additional values for sample statements that were not executed for trace tr , but due to stochastic branching or dynamic

addressing are now executed for the new trace tr' . For instance in Figure 3 Program 2, if we change the value of the last random variable "b_i" from 0 to 1, then we may have to sample values for multiple "b_j" where $j > i$ until we get $\text{tr}'(\text{"b_j"}) = 0$.

The proposed trace tr' is accepted (made current) with probability $A = \min\left(1, \frac{p(\text{tr}') \cdot Q_\alpha(\text{tr}|\text{tr}')}{p(\text{tr}) \cdot Q_\alpha(\text{tr}'|\text{tr})}\right)$, where Q_α captures the probability of the proposal process (sampling a value for α and all additional values as explained above). Then, the set of all generated traces is an approximation to the posterior distribution of the model.

In our approach, if the address comes from sample node N_k , then we can optimise the runtime of this algorithm by noting that $\log p(\text{tr}') - \log p(\text{tr}) = \sum_{j=1}^K \log p_j(\text{tr}') - \log p_j(\text{tr}) = \sum_{j \in \mathcal{J}_k \cup \{k\}} \log p_j(\text{tr}') - \log p_j(\text{tr})$. Thus, by being able to compute the factors $p_j(\text{tr}')$ and $p_j(\text{tr})$ separately, we can compute the ratio of densities $p(\text{tr}')/p(\text{tr})$ by summing fewer terms.

By designing execution contexts for the sub-programs to handle the effect of the LMH kernel, we statically enable this optimisation for programs with an unbounded number of random variables and dynamic dependencies. In the following we informally describe the implementation of two execution contexts for resampling address α at sample node N_k . A formal treatment can be found in Appendix D.1.

When executed with ForwardCtx, the sub-program is evaluated with respect to the proposed trace tr' . At `visit(ForwardCtx,  $\rho, E_0, f(\dots)$ )` we sample a new value v for address $\alpha = \rho(E_0)$ from some proposal distribution q and return v . As before, $\rho(E_0)$ denotes E_0 evaluated with respect to ρ . In the background, we accumulate the log-probability $\Delta P \leftarrow \Delta P + \log \text{pdf}_f(v; \dots)$ and proposal log-probability $\Delta Q \leftarrow \Delta Q - \log \text{pdf}_q(v; \dots)$. At the start, ΔP and ΔQ are set to 0. At `score(ForwardCtx,  $\rho, E_0, f(\dots)$ )` we propose a new value v *only* if the address $\rho(E_0)$ is not sampled for the current trace. We always update ΔP as before and only update ΔQ if we have proposed a new value. At `read(ForwardCtx,  $\rho, E_0$ )`, we simply return $\text{tr}(\rho(E_0))$. At each of the above statements we cache the program state in ρ . This introduces book-keeping cost, but as we will see in Section 5.2.1, the cost is outweighed by the performance gained from only running sub-programs.

When executed with BackwardCtx, the sub-program is evaluated with respect to the current trace tr . At `visit(BackwardCtx,  $\rho, E_0, f(\dots)$ )` we update $\Delta P \leftarrow \Delta P - \log \text{pdf}_f(v; \dots)$ and proposal log-probability $\Delta Q \leftarrow \Delta Q + \log \text{pdf}_q(v; \dots)$ with the current value $v = \text{tr}(\rho(E_0))$. At `score(BackwardCtx,  $\rho, E_0, f(\dots)$ )` the same ΔP -updates are always performed while the ΔQ -updates are only performed if the address $\rho(E_0)$ is not sampled for the *proposed* trace tr' . At statement `read(BackwardCtx,  $\rho, E_0$ )` we again return $\text{tr}(\rho(E_0))$. The program state does not have to be cached when executing with BackwardCtx.

Lastly, after executing the sub-program in both contexts, we have $A = \min(1, \exp(\Delta P + \Delta Q))$. We accept the proposed trace with probability A . In this case, we perform some book-keeping to update the inference state to the proposed trace with the cached program states. Building on the soundness of our provenance analysis and Theorem 3, we proof correctness of this approach in Appendix D.4:

COROLLARY 2. *Assume that the address expression of at most one sample node in the execution sequence evaluates to α for all traces. Let tr' be the LMH proposal at address α for tr . Then, the execution sequences of tr and tr' are equal up to (σ, M_α) , where M_α is the first and only sample node whose address expression evaluates to α . For the sliced CFG G_k , where $\iota_{\text{cfg}}(M_\alpha) = N_k$, we have*

$$\log p(\text{tr}') - \log p(\text{tr}) = \log \bar{p}_{G_k}(\text{tr}', \sigma) - \log \bar{p}_{G_k}(\text{tr}, \sigma).$$

The quantities $\log \bar{p}_{G_k}(\text{tr}', \sigma)$, and $\log Q_\alpha(\text{tr}'|\text{tr})$ are computed by ForwardCtx and the quantities $\log \bar{p}_{G_k}(\text{tr}, \sigma)$ and $\log Q_\alpha(\text{tr}|\text{tr}')$ are computed by BackwardCtx. The new trace tr' can be constructed from tr and the output of both contexts.

Table 1. Average runtime of computing a single LMH iteration in microseconds (see Section 5.2). The column "Baseline" reports the runtime when computing $p(\text{tr}')$ from scratch. The column "Static Analysis" reports the runtime when updating the old density by computing $\log p(\text{tr}') - \log p(\text{tr})$ with the generated sub-programs (see Section 5.1). The column "Fixed-Finite" reports the runtime of updating the density with a hand-written factorisation that is representative of PPLs that explicitly construct a graphical model. The speed-up in relation to computing from scratch is given in the "speed-up" and "sp.up" columns. The number of random variables defined in each model can be found in column "#RVs". The number of samples produced and the acceptance rate are reported in the N and AR column, respectively.

Model	#RVs	N	AR	Baseline	Static Analysis		Fixed-Finite	
				μs	μs	speed-up	μs	sp.up
Aircraft	∞	1e5	0.47	5.861	4.605	1.29 ↗	-	-
Bayesian Network	37	1e5	0.93	25.580	4.680	5.47 ↑↑	3.344	7.65
Captcha	∞	1e3	0.81	774.287	355.382	2.18 ↑↑	-	-
Dirichlet process	∞	1e4	0.53	89.939	42.664	2.12 ↑↑	-	-
Geometric	∞	5e5	0.62	2.419	2.475	0.97 →	-	-
GMM (fixed #clusters)	209	5e4	0.60	71.136	7.027	10.01 ↑↑	4.564	15.58
GMM (variable #clusters)	∞	5e4	0.57	74.709	9.549	7.81 ↑↑	-	-
HMM	101	1e5	0.74	33.729	24.016	1.39 ↗	2.551	13.22
Hurricane	5	1e6	0.86	2.121	1.918	1.13 →	-	-
LDA (fixed #topics)	551	1e4	0.81	209.046	24.526	8.52 ↑↑	18.341	11.40
LDA (variable #topics)	∞	1e4	0.83	218.614	25.732	8.50 ↑↑	-	-
Linear regression	102	1e5	0.53	4.477	8.006	0.56 ↓↓	7.504	0.60
Marsaglia	∞	5e5	0.93	1.699	2.159	0.79 ↘	-	-
PCFG	∞	1e5	0.75	10.220	8.437	1.21 →	-	-
Pedestrian	∞	1e5	0.37	3.709	3.645	1.02 →	-	-
Urn	∞	1e5	0.52	11.979	6.354	1.88 ↗	-	-

5.2.1 Experiment Results. To evaluate the speed-up made possible by our static factorisation, we gathered a benchmark set of 16 probabilistic programs. We specifically collected probabilistic programs from prior research that make use of while loops to define an unbounded number of random variables. References for the programs can be found in our replication package [10]. Only 5 out of the 16 models define just a finite number of random variables with fixed dependency structure. It takes less than five seconds to generate the sub-programs for all models.

For each model, we run two implementations of the LMH inference algorithm. As a baseline, the standard implementation of LMH computes the acceptance probability A by computing the density $p(\text{tr}')$ from scratch. Our optimised version of LMH leverages the sub-programs to compute $\log p(\text{tr}') - \log p(\text{tr})$ when calculating A . For the five finite models with fixed structure, we additionally implemented a factorisation that is representative of PPLs that explicitly construct a probabilistic graphical model. This factorisation enables LMH to exploit the full dependency structure of the model and serves as comparison. We have tested correctness of our LMH implementations by asserting that *exactly* the same acceptance probabilities and samples are computed.

In Table 1, we report the results of the experiment run on a M2 Pro CPU. For each model and implementation, we ran LMH with 10 random starting traces for a total of N samples and average the measurements over 10 experiment repetitions.

The first observation that we want to highlight is that computing $\log p(\text{tr}') - \log p(\text{tr})$ with our approach can lead to longer runtime if there is no structure to exploit in the model. Consider the linear regression model in which the 100 observations depend on the slope and intercept variables.

When changing the value of one of these variables, computing $\log p(\text{tr}') - \log p(\text{tr})$ requires 100 additions and 100 subtractions, while $\log p(\text{tr}')$ only takes 100 additions. The same is true for the Marsaglia model which implements a rejection sampling method for the Normal distribution such that all variables depend on each other.

The factorisation approach shines when there is a lot of structure in the models like for the Aircraft, Captcha, Dirichlet Process, and Urn models, where the runtime is decreased up to a factor of 2. It is particularly powerful, when there are latent variables that only affect independent data variables. This is the case for the Gaussian mixture model (GMM) where there is one cluster allocation variable per data point and for the Latent Dirichlet Allocation model (LDA) where there is similar dependency structure. For these models we observe 8-10x speed-ups. Note that the handwritten "fixed-finite" factorisations of the models are faster than the "static analysis" factorisations, because LMH has to do less book-keeping for them. They do not leverage additional dependency structure.

The only model where our approach fails to effectively leverage the true dependency structure is the Hidden Markov Model (HMM). Even though the current state only affects the next state, our static analysis over-approximates this structure and outputs that the current state affects all future states. Now, if we modify the program by unrolling the while loop, i.e. repeating its body fifty times, our factorisation approach results in an improved runtime of $2.605 \mu\text{s}$ (13x speed-up), because in this case the dependency structure is not over-approximated anymore due to loops. Note that the PCFG model essentially has the same structure as HMM, but can transition to a terminal state at every loop iteration. In this case, indeed the current state affects all future states and our static analysis is exact, but no significant improvements in runtime are measurable.

Lastly, we note that the Hurricane model is the only model in our benchmark with a finite number of random variables but a dynamic dependency structure (see Section 3.3.2). This model can also not be represented in a conventional Bayesian network structure, but our method is applicable and can slightly reduce the runtime.

5.2.2 Comparison to Related Methods. The approach presented in Section 5 is the first to generate optimised MCMC kernels of models with unbounded number of random variables via a static program transformation. Importantly, its correctness follows from the theoretical foundation established in Section 4. Nevertheless, there exist methods that aim to accelerate MCMC kernels for a similar class of models with other approaches. In this section, our goal is to make a fair comparison to those methods by focusing on the relative speed-ups achieved. A direct runtime comparison is not meaningful due to differences in programming languages and the implementation of inference algorithms. More related approaches are described in Section 6.

Table 2. Relative speed-up for the LMH algorithm achieved with our static approach, combinators in Gen, and the C3 algorithm in WebPPL.

Model	ours	Gen	C3	Model	ours	Gen	C3
Aircraft	1.29	1.41	1.02	Hurricane	1.13	0.51	0.93
Bayesian Network	5.47	4.14	0.93	LDA (fixed #topics)	8.52	6.50	4.97
Captcha	2.18	2.93	1.05	LDA (variable #topics)	8.50	5.92	7.26
Dirichlet process	2.12	-	1.24	Linear regression	0.56	0.43	0.44
Geometric	0.97	-	0.74	Marsaglia	0.79	-	0.53
GMM (fixed #clusters)	10.01	6.26	1.90	PCFG	1.21	-	0.79
GMM (variable #clusters)	7.81	5.45	1.60	Pedestrian	1.02	-	0.52
HMM	1.39	1.31	3.82	Urn	1.88	2.05	0.69

First, Ritchie et al. [48] presented the C3 approach to enable incrementalised re-execution of Metropolis Hastings kernels using continuations and function call caching. To be effective this method requires programs to be implemented with recursion instead of while loops. We re-implemented the models in recursive form in WebPPL [22] to report the relative speed-up achieved with C3 in Table 2. Our approach achieves better speed-ups for all models except for the HMM. As explained before, our static analysis greatly over-approximates the dependency structure of this model. Note that Ritchie et al. [48] report even more significant speed-ups for the GMM, LDA, and HMM models for larger data-set sizes. Our approach scales the same for GMM and LDA.

Further, modern probabilistic programming systems like Gen [17] include language constructs like Map, Unfold, and Recurse to model independence structure. Thus, we re-implemented LMH and the 16 models only in Gen with the so-called combinator constructs and report the results in Table 2. Gen's inference backend can only leverage the combinator constructs if the root model function is written in the *static modelling language* – a non-universal subset of the language with no control statements. This effectively abstracts away loops as Map, Unfold, or Recurse nodes in an internal graphical representation. Note that at the time of writing Gen does not implement incremental computation for the Recurse combinator, a construct required for models where the condition of the while loop depends on the body like the Geometric model. We cannot apply our general LMH implementation to those models and mark them with "-". Overall, the order of magnitude of speed-up achieved with combinators is similar to our static approach. There seems to be more overhead in Gen as it supports general programmable inference in contrast to our research PPL. We emphasise that our approach achieves these speed-ups fully automatically from source code without the need to explicitly model independence structure with language constructs.

5.3 Reducing the Variance of an ELBO Gradient Estimator in Variational Inference

Variational inference takes a different approach to MCMC methods by fitting a so-called variational distribution Q_ϕ to the posterior, where ϕ is a set of parameters. The objective is to maximise the evidence lower-bound $\text{ELBO}(\phi) = \mathbb{E}_{z \sim Q_\phi} [\log P(z) - \log Q_\phi(z)]$ via gradient ascent, where P denotes a potentially unnormalised density. In this section, we consider Black-Box Variational Inference (BBVI) [47], which estimates the gradient of the ELBO with following identity $\nabla_\phi \text{ELBO}(\phi) = \mathbb{E}_{z \sim Q_\phi} [\nabla_\phi \log Q_\phi(z) \cdot (\log P(z) - \log Q_\phi(z))]$. Unfortunately, this estimator often exhibits high variance. To reduce the variance Ranganath et al. [47] use a mean-field variational distribution $Q_\phi(z) = \prod_{i=1}^n q_i(z_i | \phi_i)$ to derive the improved estimator

$$\nabla_{\phi_i} \text{ELBO}(\phi) = \mathbb{E}_{z_{(i)} \sim Q_\phi^i} [\nabla_{\phi_i} \log q_i(z_i | \phi_i) \cdot (\log P^i(z_{(i)}) - \log q_i(z_i | \phi_i))]. \quad (10)$$

Note that the gradient is only with respect to ϕ_i and the expectation is with respect to the *Markov blanket* $z_{(i)}$ of z_i . Including z_i , this is the set of variables that directly influence z_i according to P , variables that are directly influenced by z_i – *children* of z_i – and variables that directly influence a child of z_i . Then, P^i is the density factoring out all other variables and Q_ϕ^i the variational distribution restricted to $z_{(i)}$. This reduces the variance and we will replicate this trick with our factorisation.

In our sub-program setting, we achieve the same optimisation by defining the execution context `BBVICtx`, which we formalise in Appendix D.5. First, assume that a trace `tr` is generated from a variational distribution, where each address α is sampled from $\text{tr}(\alpha) \sim q_\alpha(\text{tr}(\alpha) | \phi_\alpha)$. The `BBVICtx` executes the program with respect to `tr`. Similar to the `ForwardCtx`, it accumulates $\Delta \leftarrow \Delta + \log \text{pdf}_f(v; \dots)$ at `score(BBVICtx, \rho, E_0, f(\dots))` and returns $v = \text{tr}(\rho(E_0))$ at `read(BBVICtx, \rho, E_0)`. At visit statement `visit(BBVICtx, \rho, E_0, f(\dots))` we additionally subtract the proposal density $\Delta \leftarrow \Delta + \log \text{pdf}_f(v; \dots) - \log q_\alpha(v | \phi_\alpha)$, where we have $\alpha = \rho(E_0)$ (compare Eq. (10)). In Appendix D.5. we prove following correctness result as a consequence of Corollary 1:

COROLLARY 3. *Let Q_ϕ be a variational distribution over traces as formalised in Appendix D.5. Assume that the address expression of at most one sample node in the execution sequence evaluates to α for all traces. Under measurability assumptions given in Appendix D.5 and with p_α as in Corollary 1,*

$$\nabla_{\phi_\alpha} \mathbb{E}_{\text{tr} \sim Q_\phi} [\log p(\text{tr}) - \log Q_\phi(\text{tr})] = \mathbb{E}_{\text{tr} \sim Q_\phi} [\delta_{\text{tr}(\alpha) \neq \text{null}} \cdot \nabla_{\phi_\alpha} \log q_\alpha(\text{tr}(\alpha)|\phi_\alpha) \cdot (\log p_\alpha(\text{tr}) - \log q_\alpha(\text{tr}(\alpha)|\phi_\alpha))].$$

The quantity $\log p_\alpha(\text{tr}) - \log q_\alpha(\text{tr}(\alpha)|\phi_\alpha)$ is computed by BBVICTx.

Note that in contrast to Equation (10), where the expectation is taken explicitly with respect to the Markov blanket $z_{(i)}$, in Corollary 3 the dependencies are accounted for via $p_\alpha(\text{tr})$ (computed with a sub-program). Replacing p in the integrand with p_α reduces the variance, because p_α is constant with respect to addresses that are independent from a change to the trace at address α .

5.3.1 Experiment Results. To evaluate the approach we have implemented our BBVI optimisation and compare the improved estimator against the standard estimator with respect to the average gradient variance across all parameters. In 10 experiment repetitions, we performed 1000 optimisation steps and used 100 samples to estimate the gradient in each step. In Table 3, we report our results, where we omit programs with random variables that have stochastic support dimension and are incompatible with BBVI.

We compare against Pyro's TraceGraph_ELBO variance reduction technique, which follows Schulman et al. [53] and makes use of dynamic data provenance analysis. This comes with the benefit of having no over-approximation issues, but also comes with the drawback of not being able to support programs with control flow dependencies. For Pyro we decreased the number of samples to 100 and number of repetitions to 1 due to long runtime. In Table 3, we see that for programs with only data dependencies we achieve comparable variance reduction with lower runtime penalty. Furthermore, with our approach we were able to reduce the variance for two models with control flow dependencies Aircraft and Marsaglia. Note that for the same reasons as in LMH, the static optimisation is only effective for the unrolled version of the HMM.

Table 3. Average variance of the standard and improved ELBO gradient estimator in BBVI. The variance is averaged over all parameters, 1000 iterations, and 100 samples per iteration. The "cost" column reports the factor by which the runtime increased for the improved estimator. For reference, we include the variance reduction factor achieved by Pyro's TraceGraph_ELBO, where unsupported programs are denoted with "-".

Model	Baseline		Static Analysis		Pyro		
	avg. var.	avg. var.	red. factor	cost	red. factor	cost	
Aircraft	7.922e+03	7.969e+01	99.29	↓↓	1.89	-	-
Bayesian Network	4.160e+02	4.371e+00	95.06	↓↓	1.94	95.09	7.52
Geometric	1.634e-01	1.726e-01	0.94	→	0.53	-	-
GMM (fixed #clusters)	8.043e+09	1.014e+06	8092.80	↓↓	2.26	9848.79	8.69
HMM	2.116e+03	1.279e+03	1.65	→	7.49	1254.24	8.67
HMM (no loop)	2.116e+03	2.349e+00	901.01	↓↓	1.50	1254.24	8.64
Hurricane	3.362e-01	3.609e-01	0.93	→	0.81	-	-
LDA (fixed #topics)	7.229e+04	1.172e+00	61646.02	↓↓	5.92	59986.61	11.00
Linear regression	3.350e+05	3.341e+05	1.00	→	1.14	1.00	14.39
Marsaglia	7.274e-02	1.283e-02	5.38	↘	0.51	-	-
PCFG	8.838e-03	1.989e-02	0.56	→	0.94	-	-
Pedestrian	3.108e+06	3.111e+06	1.00	→	2.01	-	-

5.4 Iteratively Running Sequential Monte Carlo

Lastly, we briefly describe how our slicing technique can be adapted to the setting of sequential Monte Carlo (SMC) [13], which yet again takes a different approach to inference. In SMC a number of traces tr_n , so-called *particles*, are evaluated in parallel. Data points are added iteratively to the inference process and the particles are resampled according to their data likelihood. More precisely at time t we have data set $\mathcal{D}_{1:t}$ and define the corresponding density p_t . At inference step t , the particle tr is logarithmically weighted according to $\log p_t(\text{tr}_n) - \log p_{t-1}(\text{tr}_n)$ and the particles are resampled (duplicated or discarded) with respect to these weights.

A "naive" implementation of SMC would compute $\log p_t(\text{tr}_n)$ from scratch, while an iterative implementation is able to compute $\log p_t(\text{tr}_n) - \log p_{t-1}(\text{tr}_n)$ directly. In probabilistic programming this is often realised by some mechanism that enables the interruption and continuation of program execution. We enable this optimisation by slightly adapting the slicing method of Section 5.1 to generate sub-programs which compute the difference directly.

Namely, for sample node N_k let G_k^{SMC} be the CFG that contains node N if there exists a sample node N_j such that N lies on a path from N_k to N_j which contains no other sample nodes. Again, N_k will be the successor of the start node of G_k^{SMC} , but *all* other samples nodes will have the end node of G_k^{SMC} as sole successor. As we only care about continuing execution from N_k to the next sample node, this construction is independent of the provenance analysis and Theorem 2.

In our implementation, we again make use of an evaluation context which simply accumulates the density like the standard semantics. In Appendix D.6, we prove correctness:

PROPOSITION 5. *Let G be a CFG such that for initial program state σ_0 and trace tr , we have execution sequence $(\sigma_0, \text{START}) \xrightarrow{\text{tr}} \dots \xrightarrow{\text{tr}} (\sigma_i, M_i) \xrightarrow{\text{tr}} \dots \xrightarrow{\text{tr}} (\sigma_l, \text{END})$ in its unrolled version H . Let $(\sigma'_1, M'_1) \xrightarrow{\text{tr}} \dots \xrightarrow{\text{tr}} (\sigma'_2, M'_2)$ be a sub-sequence such that $\iota_{\text{cfg}}(M'_1) = N_k$ and $\iota_{\text{cfg}}(M'_2) = N_j$ are the only sample nodes. Then, the execution sequence in the unrolled version of the sliced CFG G_k^{SMC} matches the sub-sequence: $(\sigma'_1, \text{START}) \xrightarrow{\text{tr}} (\sigma'_1, M'_1) \xrightarrow{\text{tr}} \dots \xrightarrow{\text{tr}} (\sigma'_2, M'_2) \xrightarrow{\text{tr}} (\sigma'_2, \text{END})$.*

Proposition 5 allows us to evolve the execution sequence of a particle tr_n from one sample node $N_t^n = N_k$ to the next $N_{t+1}^n = N_j$ by executing the corresponding sliced CFG G_k^{SMC} . The weight of the particle is updated with $\log p_t(\text{tr}_n) - \log p_{t-1}(\text{tr}_n) = \log \sigma'_2(\mathbf{p}) - \log \sigma'_1(\mathbf{p})$ and particles may be resampled even if the execution sequences were stopped at different sample nodes N_t^n [36].

Table 4. Average runtime of naive versus iterative implementations of SMC with 100 particles. For reference, we report the speed-up achieved by WebPPL's CPS transformation.

Model	#Data	Baseline	Static Analysis		WebPPL
		ms	ms	speed-up	speed-up
Dirichlet process	50	238.323	22.374	10.57 ↑↑	8.96
GMM (fixed #clusters)	100	166.480	31.260	5.26 ↑↑	9.23
GMM (variable #clusters)	100	189.529	31.263	5.95 ↑↑	9.18
HMM	50	31.951	7.394	4.13 ↑↑	7.66
HMM (no loop)	50	49.649	7.420	6.63 ↑↑	6.79
LDA (fixed #topics)	262	1319.553	397.074	3.33 ↑↑	7.25
LDA (variable #topics)	262	1412.182	410.781	3.44 ↑↑	8.18

5.4.1 Experiment Results. To measure the performance gains achievable with our static approach, we compare the runtime of the iterative implementation of SMC based on the generated sub-programs against a naive implementation which computes $\log p_t(\text{tr})$ from scratch. In Table 4 we

report the speed-ups on the subset of models which can make use of iteratively adding data points in inference. For all models we set the number of particles to 100 and repeated the experiment 10 times. The measured speed-ups are significant and range from 3x - 10x. For reference, we benchmarked the runtimes achieved with WebPPL's continuation passing style (CPS) approach against a naive SMC implementation in JavaScript. For most models this resulted in greater speed-ups compared to our static approach. This is due to the fact that, like in Section 5.2, in our approach computational overhead is introduced by caching and copying programs states when resampling particles.

6 Related Work

6.1 Semantics of Probabilistic Programming Languages

One of the first works to formally define the semantics of a probabilistic programming language was Kozen [32]. Derivatives of this semantics can be found throughout literature, for example see Hur et al. [30], Dahlqvist and Kozen [18], or Barthe et al. [2]. We give a brief (and incomplete) overview of existing semantic models for PPLs.

In recent years, a lambda calculus for probabilistic programming was presented by Borgström et al. [9]. This work and a domain theoretic approach for probabilistic programming [56] had an influence on the development of the statistical PCF [38] (programming computable functions), which was used to prove differentiability of terminating probabilistic programs. Probabilistic programming was also examined from a categorical perspective [27] and the semantics of higher order probabilistic programs is also researched [18, 54].

As mentioned, the semantics presented in this work extend those of Gorinova et al. [24] which formalises the core language constructs of Stan by interpreting a program as a function over a fixed set of finite variables. In our semantics, we interpret programs as functions over traces which are mappings from addresses to values similar to dictionaries. Another dictionary based approach to probabilistic programming can be found in Cusumano-Towner et al. [15] and denotational semantics based on trace types are introduced in Lew et al. [34]. To the best of our knowledge there are only two existing semantic models that support user-defined addresses [33, 35]. Note that denotational semantics of a simplified version of the address-based PPL Gen are given in Cusumano-Towner [16].

Furthermore, the connection of simple probabilistic programs to Bayesian networks is known [57]. Borgström et al. [8] consider a small imperative language and give semantics in terms of factor graphs – a concept related to Bayesian networks. Sampson et al. [52] compile probabilistic programs to Bayesian networks by treating loops as black-box functions in order to verify probabilistic assertion statements. Previous work extended Bayesian networks to so-called contingent Bayesian networks in order to support models with an infinite number of random variables or cyclic dependencies [42], but they cannot be straightforwardly applied to PPLs with user-labelled sample statements and while loops. Paquet [46] presented a interpretation of probabilistic programs in game semantics, which can be seen as a refinement of Bayesian networks. Most recently, Faggian et al. [19] introduced a higher-order functional language for discrete Bayesian networks.

6.2 Graphical Representations, Optimisations, and Dependency Analysis

The advantages of representing a probabilistic model graphically are well-understood [31]. For this reason, some PPLs like PyMC [51], Factorie [40], or Infer.NET [43] require the user to explicitly define their model as a graph. This is not possible if we want to support while loops in our programs. There are also other graphical representations that allow for optimised inference like binary decision diagrams [28] or sum-product networks [50]. Venture [39] represents traces graphically to facilitate

efficient recomputation of subparts of the program. Similar to Gen [17], Pyro [6] implements the `plate` and `markov` constructs to model independence and reduce variance in variational inference.

Modifying the Metropolis Hastings algorithm to exploit the structure of the model is also common practice. Wu et al. [59] optimised BLOG to dynamically keep track of the Markov blanket of a variable among other improvements. For their Metropolis Hastings implementation, they reported significant speed-ups for the Hurricane model, instances of the Urn model, and GMMs with unknown number of clusters. The Shred system [60] traces Church [21] programs and slices them to identify the minimal computation needed to compute the MH acceptance probability, and translates them to C++. However, their approach comes with high compilation cost, is not proven correct nor open-source. Hur et al. [29] developed a different theory for slicing probabilistic programs. In their unpublished implementation, they observe significant runtime improvements if statements irrelevant to the return expression are removed from the program. In contrast, we slice programs with respect to the dependencies between sample statements in the program and there are no redundancies in the programs of our benchmark set. Castellan and Paquet [11] present a theoretical foundation for incremental computation in the Metropolis Hastings algorithm by graphically modelling the data-dependencies between sample statements. However, they only consider a first-order probabilistic programs and did not implement their approach.

Furthermore, there are many more optimisation opportunities to reduce the variance in variational inference like control variates [45] or path-derivatives [49]. For a review on advances in variational inference see Zhang et al. [61]. Also, there are many methods to facilitate iterative implementations of SMC: WebPPL [22] and Anglican [55] use a continuation-passing-style transform; Turing [20] uses a tape of instructions; some PPLs like Birch [44] require the user to explicitly write the model in an iterative fashion.

Lastly, the probabilistic dependency structure of programs has also been exploited for applications different from optimising inference. Baah et al. [1] leverage the structure for fault diagnosis and Bernstein et al. [5] extract the factor graph of programs to automate model checking for models implemented in Stan. In general, static analysis for probabilistic programming is an active research field. For instance, Lee et al. [33] developed a static analysis to verify so-called guide programs for stochastic variational inference in Pyro. Gorinova et al. [25] developed an information flow type system and program transformation to automatically marginalise out discrete variables in Stan. Another example is the extension of the Stan compiler with a pedantic mode that statically catches common pitfalls in probabilistic programming [4]. Recently, a framework for statically analysing probabilistic programs in a language-agnostic manner was presented [7]. For a 2019 survey on static analysis for probabilistic programming see Bernstein [3].

7 Conclusion

In this work, we addressed the open question to what extent a probabilistic program with user-labelled sample statements and while loops can be represented graphically. We built on existing formal semantics to support these language features. We developed a sound static analysis that approximates the dependency structure of random variables in the program and used this structure to factorise the implicitly defined program density. The factorisation is equivalent to the known Bayesian network factorisation in a restricted case, but novel for programs with dynamic addresses and loops. Lastly, we demonstrated that by generating a sub-program for each factor based on the static dependency structure, we can 1) speed-up a single-site Metropolis Hastings algorithm; 2) reduce gradient variance in variational inference; 3) facilitate an iterative implementation of sequential Monte Carlo. This work is the first to make these optimisations statically available for the considered program class. We proved correctness and empirically showed that our approach performs comparably or better than existing methods.

Data-Availability Statement

The source code to reproduce the experiments of Section 5 is made available on Zenodo [10] and for reuse on <https://github.com/ipa-lab/PPLStaticFactor>. The shared repository includes the implementation of the static provenance analysis (Section 3.2) for a research PPL, the generation of sub-programs according to the dependency structure (Section 5), as well as a standard and optimised implementations of the LMH, BBVI, and SMC algorithms. It also includes the 16 benchmark programs (Section 5.2.1) written in our research PPL, with and without combinators in Gen, in recursive fashion in WebPPL, and in Pyro.

Acknowledgments

The authors gratefully acknowledge the financial support under the scope of the COMET program within the K2 Center “Integrated Computational Material, Process and Product Engineering (IC-MPPE)” (Project No 886385). This program is supported by the Austrian Federal Ministries for Economy, Energy and Tourism (BMWET) and for Innovation, Mobility and Infrastructure (BMIMI), represented by the Austrian Research Promotion Agency (FFG), and the federal states of Styria, Upper Austria and Tyrol. Additionally, this research has been partially funded by the Austrian Science Fund (FWF) under grant 10.55776/PIN3275223 (<https://doi.org/10.55776/PIN3275223>).

References

- [1] George K Baah, Andy Podgurski, and Mary Jean Harrold. 2008. The probabilistic program dependence graph and its application to fault diagnosis. In *Proceedings of the 2008 international symposium on Software testing and analysis*. 189–200. doi:10.1145/1390630.1390654
- [2] Gilles Barthe, Joost-Pieter Katoen, and Alexandra Silva. 2020. *Foundations of probabilistic programming*. Cambridge University Press. doi:10.1017/9781108770750
- [3] Ryan Bernstein. 2019. Static analysis for probabilistic programs. *arXiv preprint arXiv:1909.05076* (2019).
- [4] Ryan Bernstein. 2023. *Abstractions for Probabilistic Programming to Support Model Development*. Ph. D. Dissertation. Columbia University. doi:10.7916/D5Y3-6F17
- [5] Ryan Bernstein, Matthijs Vákár, and Jeannette Wing. 2020. Transforming probabilistic programs for model checking. In *Proceedings of the 2020 ACM-IMS on Foundations of Data Science Conference*. 149–159. doi:10.1145/3412815.3416896
- [6] Eli Bingham, Jonathan P Chen, Martin Jankowiak, Fritz Obermeyer, Neeraj Pradhan, Theofanis Karaletsos, Rohit Singh, Paul Szerlip, Paul Horsfall, and Noah D Goodman. 2019. Pyro: Deep universal probabilistic programming. *The Journal of Machine Learning Research* 20, 1 (2019), 973–978.
- [7] Markus Böck, Michael Schröder, and Jürgen Cito. 2024. Language-Agnostic Static Analysis of Probabilistic Programs. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 78–90. doi:10.1145/3691620.3695031
- [8] Johannes Borgström, Andrew D Gordon, Michael Greenberg, James Margetson, and Jurgen Van Gael. 2011. Measure transformer semantics for Bayesian machine learning. In *Programming Languages and Systems: 20th European Symposium on Programming, ESOP 2011, Saarbrücken, Germany, March 26–April 3, 2011. Proceedings 20*. Springer, 77–96. doi:10.1007/978-3-642-19718-5_5
- [9] Johannes Borgström, Ugo Dal Lago, Andrew D. Gordon, and Marcin Szymczak. 2016. A lambda-calculus foundation for universal probabilistic programming. In *Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming, ICFP 2016, Nara, Japan, September 18–22, 2016*, Jacques Garrigue, Gabriele Keller, and Eijiro Sumii (Eds.). ACM, 33–46. doi:10.1145/2951913.2951942
- [10] Markus Böck. 2026. *Static Factorisation of Probabilistic Programs With User-Labelled Sample Statements and While Loops*. doi:10.5281/zenodo.18482696
- [11] Simon Castellan and Hugo Paquet. 2019. Probabilistic programming inference via intensional semantics. In *Programming Languages and Systems: 28th European Symposium on Programming, ESOP 2019, Prague, Czech Republic, April 6–11, 2019. Proceedings 28*. Springer, 322–349. doi:10.1007/978-3-030-17184-1_12
- [12] Yutian Chen, Vikash Mansinghka, and Zoubin Ghahramani. 2014. Sublinear approximate inference for probabilistic programs. *stat* 1050 (2014), 6.
- [13] Nicolas Chopin, Omiros Papaspiliopoulos, et al. 2020. *An introduction to sequential Monte Carlo*. Vol. 4. Springer. doi:10.1007/978-1-4757-3437-9_1
- [14] Keith D Cooper and Linda Torczon. 2011. *Engineering a compiler*. Elsevier.

- [15] Marco Cusumano-Towner, Alexander K Lew, and Vikash K Mansinghka. 2020. Automating involutive MCMC using probabilistic and differentiable programming. *arXiv preprint arXiv:2007.09871* (2020).
- [16] Marco Francis Cusumano-Towner. 2020. *Gen: a high-level programming platform for probabilistic inference*. Ph.D. Dissertation. Massachusetts Institute of Technology.
- [17] Marco F Cusumano-Towner, Feras A Saad, Alexander K Lew, and Vikash K Mansinghka. 2019. Gen: a general-purpose probabilistic programming system with programmable inference. In *Proceedings of the 40th acm sigplan conference on programming language design and implementation*. 221–236. doi:10.1145/3314221.3314642
- [18] Fredrik Dahlqvist and Dexter Kozen. 2020. Semantics of higher-order probabilistic programs with conditioning. *Proc. ACM Program. Lang.* 4, POPL (2020), 57:1–57:29. doi:10.1145/3371125
- [19] Claudia Faggian, Daniele Pautasso, and Gabriele Vanoni. 2024. Higher Order Bayesian Networks, Exactly. *Proceedings of the ACM on Programming Languages* 8, POPL (2024), 2514–2546. doi:10.1145/3632926
- [20] Hong Ge, Kai Xu, and Zoubin Ghahramani. 2018. Turing: a language for flexible probabilistic inference. In *International conference on artificial intelligence and statistics*. PMLR, 1682–1690.
- [21] Noah D. Goodman, Vikash K. Mansinghka, Daniel M. Roy, Kallista A. Bonawitz, and Joshua B. Tenenbaum. 2008. Church: a language for generative models. In *UAI 2008, Proceedings of the 24th Conference in Uncertainty in Artificial Intelligence, Helsinki, Finland, July 9-12, 2008*, David A. McAllester and Petri Myllymäki (Eds.). AUAI Press, 220–229.
- [22] Noah D Goodman and Andreas Stuhlmüller. 2014. The Design and Implementation of Probabilistic Programming Languages. <http://dippl.org>. Accessed: 2025-3-20.
- [23] Andrew D. Gordon, Thomas A. Henzinger, Aditya V. Nori, and Sriram K. Rajamani. 2014. Probabilistic programming. In *Proceedings of the on Future of Software Engineering, FOSE 2014, Hyderabad, India, May 31 - June 7, 2014*, James D. Herbsleb and Matthew B. Dwyer (Eds.). ACM, 167–181. doi:10.1145/2593882.2593900
- [24] Maria I. Gorinova, Andrew D. Gordon, and Charles Sutton. 2019. Probabilistic programming with densities in SlicStan: efficient, flexible, and deterministic. *Proc. ACM Program. Lang.* 3, POPL (2019), 35:1–35:30. doi:10.1145/3290348
- [25] Maria I Gorinova, Andrew D Gordon, Charles Sutton, and Matthijs Vákár. 2022. Conditional independence by typing. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 44, 1 (2022), 1–54. doi:10.1145/3490421
- [26] John L Hennessy and David A Patterson. 2011. *Computer architecture: a quantitative approach*. Elsevier.
- [27] Chris Heunen, Ohad Kammar, Sam Staton, and Hongseok Yang. 2017. A convenient category for higher-order probability theory. In *2017 32nd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*. IEEE, 1–12. doi:10.1109/LICS.2017.8005137
- [28] Steven Holtzen, Guy Van den Broeck, and Todd Millstein. 2020. Scaling exact inference for discrete probabilistic programs. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020), 1–31. doi:10.1145/3428208
- [29] Chung-Kil Hur, Aditya V Nori, Sriram K Rajamani, and Selva Samuel. 2014. Slicing probabilistic programs. *ACM SIGPLAN Notices* 49, 6 (2014), 133–144. doi:10.1145/2594291.2594303
- [30] Chung-Kil Hur, Aditya V Nori, Sriram K Rajamani, and Selva Samuel. 2015. A provably correct sampler for probabilistic programs. In *35th IARCS Annual Conference on Foundations of Software Technology and Theoretical Computer Science (FSTTCS 2015)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik. doi:10.4230/LIPICS.FSTTCS.2015.475
- [31] Daphne Koller and Nir Friedman. 2009. *Probabilistic graphical models: principles and techniques*. MIT press.
- [32] Dexter Kozen. 1979. Semantics of probabilistic programs. In *20th Annual Symposium on Foundations of Computer Science (sfcs 1979)*. IEEE, 101–114. doi:10.1109/SFCS.1979.38
- [33] Wonyeol Lee, Hangyeol Yu, Xavier Rival, and Hongseok Yang. 2019. Towards verified stochastic variational inference for probabilistic programs. *Proceedings of the ACM on Programming Languages* 4, POPL (2019), 1–33. doi:10.1145/3371084
- [34] Alexander K Lew, Marco F Cusumano-Towner, Benjamin Sherman, Michael Carbin, and Vikash K Mansinghka. 2019. Trace types and denotational semantics for sound programmable inference in probabilistic languages. *Proceedings of the ACM on Programming Languages* 4, POPL (2019), 1–32. doi:10.1145/3371087
- [35] Alexander K Lew, Matin Ghavamizadeh, Martin C Rinard, and Vikash K Mansinghka. 2023. Probabilistic Programming with Stochastic Probabilities. *Proceedings of the ACM on Programming Languages* 7, PLDI (2023), 1708–1732. doi:10.1145/3591290
- [36] Daniel Lundén, Johannes Borgström, and David Broman. 2021. Correctness of Sequential Monte Carlo Inference for Probabilistic Programming Languages.. In *ESOP*. 404–431. doi:10.1007/978-3-030-72019-3_15
- [37] David J Lunn, Andrew Thomas, Nicky Best, and David Spiegelhalter. 2000. WinBUGS—a Bayesian modelling framework: concepts, structure, and extensibility. *Statistics and computing* 10 (2000), 325–337. doi:10.1023/A:1008929526011
- [38] Carol Mak, C-H Luke Ong, Hugo Paquet, and Dominik Wagner. 2021. Densities of almost surely terminating probabilistic programs are differentiable almost everywhere. In *Programming Languages and Systems: 30th European Symposium on Programming, ESOP 2021, Luxembourg City, Luxembourg, March 27–April 1, 2021, Proceedings 30*. Springer International Publishing, 432–461. doi:10.1007/978-3-030-72019-3_16
- [39] Vikash K Mansinghka, Ulrich Schaechtle, Shivam Handa, Alexey Radul, Yutian Chen, and Martin Rinard. 2018. Probabilistic programming with programmable inference. In *Proceedings of the 39th ACM SIGPLAN Conference on*

- Programming Language Design and Implementation*. 603–616. doi:10.1145/3192366.3192409
- [40] Andrew McCallum, Karl Schultz, and Sameer Singh. 2009. Factorie: Probabilistic programming via imperatively defined factor graphs. *Advances in Neural Information Processing Systems* 22 (2009).
 - [41] Brian Milch, Bhaskara Marthi, Stuart Russell, David A. Sontag, Daniel L. Ong, and Andrey Kolobov. 2005. BLOG: Probabilistic Models with Unknown Objects. In *IJCAI-05, Proceedings of the Nineteenth International Joint Conference on Artificial Intelligence, Edinburgh, Scotland, UK, July 30 - August 5, 2005*, Leslie Pack Kaelbling and Alessandro Saffiotti (Eds.). Professional Book Center, 1352–1359.
 - [42] Brian Milch, Bhaskara Marthi, David Sontag, Stuart Russell, Daniel L Ong, and Andrey Kolobov. 2005. Approximate inference for infinite contingent Bayesian networks. In *International Workshop on Artificial Intelligence and Statistics*. PMLR, 238–245.
 - [43] Tom Minka. 2012. Infer. NET 2.5. <http://research.microsoft.com/infernet> (2012).
 - [44] Lawrence M Murray and Thomas B Schön. 2018. Automated learning with a probabilistic programming language: Birch. *Annual Reviews in Control* 46 (2018), 29–43. doi:10.1016/J.ARCONTROL.2018.10.013
 - [45] John Paisley, David Blei, and Michael Jordan. 2012. Variational Bayesian inference with stochastic search. *arXiv preprint arXiv:1206.6430* (2012).
 - [46] Hugo Paquet. 2021. Bayesian strategies: probabilistic programs as generalised graphical models. In *Programming Languages and Systems: 30th European Symposium on Programming, ESOP 2021, Luxembourg City, Luxembourg, March 27–April 1, 2021, Proceedings 30*. Springer International Publishing, 519–547. doi:10.1007/978-3-030-72019-3_19
 - [47] Rajesh Ranganath, Sean Gerrish, and David Blei. 2014. Black box variational inference. In *Artificial intelligence and statistics*. PMLR, 814–822.
 - [48] Daniel Ritchie, Andreas Stuhlmüller, and Noah Goodman. 2016. C3: Lightweight incrementalized MCMC for probabilistic programs using continuations and callsite caching. In *Artificial Intelligence and Statistics*. PMLR, 28–37.
 - [49] Geoffrey Roeder, Yuhuai Wu, and David K Duvenaud. 2017. Sticking the landing: Simple, lower-variance gradient estimators for variational inference. *Advances in Neural Information Processing Systems* 30 (2017).
 - [50] Feras A Saad, Martin C Rinard, and Vikash K Mansinghka. 2021. SPPL: probabilistic programming with fast exact symbolic inference. In *Proceedings of the 42nd acm sigplan international conference on programming language design and implementation*. 804–819. doi:10.1145/3453483.3454078
 - [51] John Salvatier, Thomas V Wiecki, and Christopher Fonnesbeck. 2016. Probabilistic programming in Python using PyMC3. *PeerJ Computer Science* 2 (2016), e55. doi:10.7717/PEERJ-CS.55
 - [52] Adrian Sampson, Pavel Panchekha, Todd Mytkowicz, Kathryn S McKinley, Dan Grossman, and Luis Ceze. 2014. Expressing and verifying probabilistic assertions. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 112–122. doi:10.1145/2594291.2594294
 - [53] John Schulman, Nicolas Heess, Theophane Weber, and Pieter Abbeel. 2015. Gradient estimation using stochastic computation graphs. *Advances in neural information processing systems* 28 (2015).
 - [54] Sam Staton, Hongseok Yang, Frank Wood, Chris Heunen, and Ohad Kammar. 2016. Semantics for probabilistic programming: higher-order functions, continuous distributions, and soft constraints. In *Proceedings of the 31st Annual ACM/IEEE Symposium on Logic in Computer Science*. 525–534. doi:10.1145/2933575.2935313
 - [55] David Tolpin, Jan-Willem van de Meent, and Frank Wood. 2015. Probabilistic programming in Anglican. In *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2015, Porto, Portugal, September 7-11, 2015, Proceedings, Part III 15*. Springer, 308–311. doi:10.1007/978-3-319-23461-8_36
 - [56] Matthijs Vákár, Ohad Kammar, and Sam Staton. 2019. A domain theory for statistical probabilistic programming. *Proceedings of the ACM on Programming Languages* 3, POPL (2019), 1–29. doi:10.1145/3290349
 - [57] Jan-Willem van de Meent, Brooks Paige, Hongseok Yang, and Frank Wood. 2018. An introduction to probabilistic programming. *arXiv preprint arXiv:1809.10756* (2018).
 - [58] David Wingate, Andreas Stuhlmüller, and Noah Goodman. 2011. Lightweight implementations of probabilistic programming languages via transformational compilation. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*. JMLR Workshop and Conference Proceedings, 770–778.
 - [59] Yi Wu, Lei Li, Stuart Russell, and Rastislav Bodik. 2016. Swift: Compiled Inference for Probabilistic Programming Languages. In *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence, IJCAI 2016, New York, NY, USA, 9-15 July 2016*, Subbarao Kambhampati (Ed.). IJCAI/AAAI Press, 3637–3645.
 - [60] Lingfeng Yang, Patrick Hanrahan, and Noah Goodman. 2014. Generating efficient MCMC kernels from probabilistic programs. In *Artificial Intelligence and Statistics*. PMLR, 1068–1076.
 - [61] Cheng Zhang, Judith Bütepage, Hedvig Kjellström, and Stephan Mandt. 2018. Advances in variational inference. *IEEE transactions on pattern analysis and machine intelligence* 41, 8 (2018), 2008–2026. doi:10.1109/TPAMI.2018.2889774

Received 2025-10-08; accepted 2026-02-17